

11/07/24

Structured Query Language

SQL is a language which is used to communicate with database.

Data is a raw fact which describes the attributes of an entity.

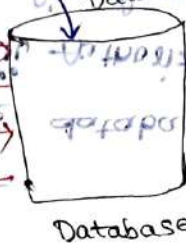
Attribute :- property

Entity :- object
 data → document
 values → raw fact

Database :- Data base is a place/contains/medium which is used to store the data in a systematic organised manner.

CRUD :- In database we are performing crud operations.

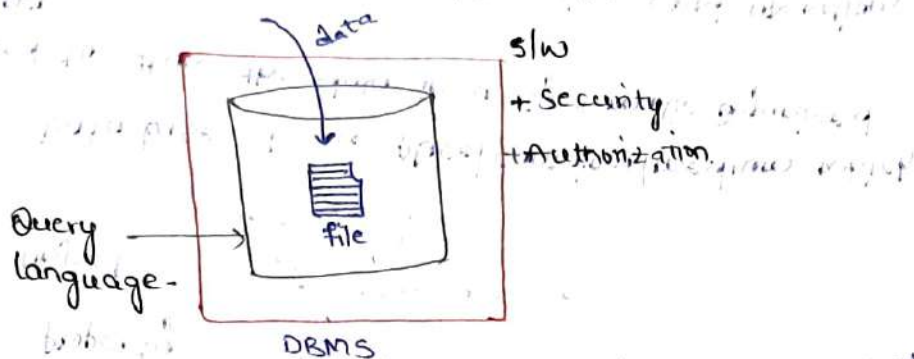
- C → create/insert
- R → Read/Retrieve
- U → update/modify
- D → delete/drop



Attributes → traits/details
 Entity → object
 living or nonliving thing

* DBMS :- Database Management System

1. DBMS is a software, it maintains the database.
2. It provides two ^{important} features, i. Security ii. Authorization.
3. It stores the data in the form of file format.



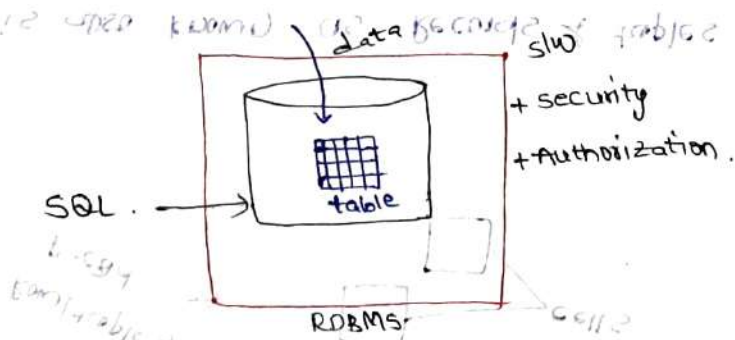
4. We can communicate/interact with database management system, by using query language.

* RDBMS :- Relational database management System

1. RDBMS is a kind of DBMS software, which can store

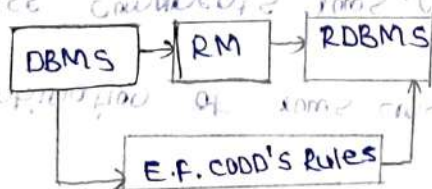
everything in the form of tables.

2. We can communicate or interact with RDBMS by using SQL



Relational model

1. Relational model was designed by E.F. Codd (Edgar Frank 'Tend' Codd)
2. In relational model everything ^{will store} in the form of rows & columns



3. Any DBMS following relational model (RM) to become RDBMS

→ DBMS maintains the database in any organisation

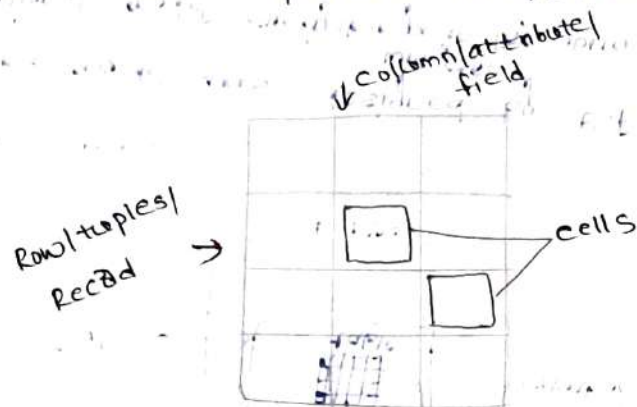
→ features → Security
→ Authorization

Edgar Frank 'Tend'
Codd - Computer Scientist

4. Any DBMS follows E.F. COOD to become a RDBMS.

* Table :-

- It is a combination of rows and columns
- It has three components rows, columns & cells



Rows:-

- Row is also known as Records & tuples
- A Row is used to represent properties of an individual entity.

Column:-

- Columns are also known as attributes/fields.
- A Column is used to represent properties of all the entities.

Cell:-

- Cell is the smallest unit of table, in which it stores the data.
- The intersection of rows and columns generate these cells.

DBMS vs RDBMS

	DBMS	RDBMS
1	DBMS has NO relationship between the data	Data is stored in the form of tables which are related to each other
2	Normalization is not present	Normalization is present

Assignment

3. DBMS does not support distributed database.	ROBMS supports distributed database.	distributed database data is not limited to one system
4. It deals with small quantity of data.	It deals with large amount of data.	
5. Data redundancy is common in this model.	It deals w/ keys and indexes do not allow data redundancy.	redundancy → repetition
6. It is used for small organisation and deal with small data.	It is used to handle large amount of data.	
7. Security is less.	More Security measures provided.	
8. It supports single user.	It supports multiple users.	
9. low software and hardware necessities.	Higher software and hardware necessities.	

Rules 10E.F. CODD

1. Data entered into a cell must be single value (atomic)

Ex: Students

SID	SName	P.No
1	DINGI	100
2	DINGA	108
3	MANGI	320

SID	SName	R.No1	PH.No2
1	DINGI	100	
2	DINGA	108	
3	MANGI	320	420

2. In RDBMS Everything stores in the form table including meta data

Meta data:- details about a data

meta table	img's name	Size	Format	Resolution	EID	ENAME	E PHOTO
	dimg'simg	1Mb	jpeg	720x1080	1	DINGI	
	dimg'simg	4mb	jpeg	720x1080	2	DINGA	
	mangisimg	2Mb	jpg	320x620	3	MANGI	

img's name: dimg'simg
Size: 1mb
format: jpeg
Resolution: 720x1080

- Metadata are store in the metatable.
- (meta data's) metatables are autogenerated.

3. According to E.F CODD we can store the data in multiple tables, if necessary we can establish connection between the tables by using key attribute.

4. The data entering into the table can be verified in two steps:-

→ By assigning datatypes.

→ By assigning constraints.

Note - datatypes is mandatory, constraints are optional.

Datatype:-

It is used to determine what kind of data one store in the particular memory location.

Types of datatype:-

i) varchar/varchar2 (CAPITAL LETTER)

ii) char

iii) DATE & CDD

iv) TIMESTAMP

v) NUMBER

vi) LARGE OBJECT

→ character large object

→ Binary large object

Note: SQL is not a case sensitive language.

CHAR:- 'A-Z', 'a-z', '0-9', special characters (!, \$, @ etc.)

→ Characters must always be enclosed within single quotes.

→ Characters are case sensitive.

→ When ever we use character datatype we must

mention size.

Syntax:- CHAR(size)

Size:- It determines number of characters that can be stored.

→ The maximum number of characters that can be stored is

2000ch.

→ It follows fixed length memory allocation.

Ex:- 'MEENU'

M	E	E	N	U					
1	2	3	4	5	6	7	8	9	10

← Used memory →

Name
varchar2
SA2 - unused
any - unused/waste

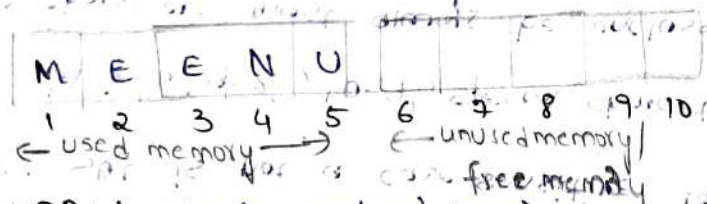
- ii, VARCHAR :- 'A-z', 'a-z', '0-9', special characters (\$, @, #, ... etc)
- Varchars always be enclosed within single quotes
- Varchars are case sensitive
- Whenever we use Varchars datatype, we must mention

Size (Syntax :- Varchar (Size))

Size :- It determines number of Varcharacters that can be stored

- The maximum number of (changes) Varchar that can be stored is 2000 ch.
- It follows variable length in memory allocation

EX :- 'MEENU'



Name
Varchar(10)
SAB - free memory
Ajay - free memory

iii, VARCHAR2 :- It is updated version of VARCHAR here upto 4000ch we can store

CHAR	VARCHAR
1. CHAR datatype is used to store character strings of fixed length	VARCHAR datatype is used to store character strings of variable length
2. In CHAR, If the length of the string is less than set or fixed-length then it is padded with extra memory space.	In VARCHAR, If the length of the string is less than the set or fixed-length then it will store as it is without padded with extra memory spaces.
3. CHAR stands for character	VARCHAR stands for variable character
4. Storage size of CHAR datatype is equal to n bytes i.e. set length	The storage size of the VARCHAR datatype is equal to the actual length of the entered string in bytes

Assignment

5. We should use the CHAR datatype when we expect the data values in a column are of the same length.

CHAR takes 1 byte for each character

Better performance than

VARCHAR

We should use the VARCHAR datatype when we expect the data values in a column are of variable length.

VARCHAR takes 1 byte for each character and some extra bytes for holding length information

performance is not good as compared to CHAR

17/12/24

DATE :-

→ We can store the DATE in (Oracle) Specified format

Syntax :- DATE

Format :- 'DD-MM-YY' or 'DD-MM-YYYY'

Ex :-

'21-MAR-24' or '21-MAR-1924'
Current Past

TIME STAMP :-

→ We can store the time along with date in specified format

Syntax :- TIME STAMP

Format :-

'HH:MM:SS.FS' YYYY-MM-DD'

NUMBER :-

It is used to store numeric values

Syntax :- NUMBER (Precision, scale)

↓
1 to 38

↓
-84 to 127

Precision :- It is used to determine number of digits to

store integer values

The range of precision is 1 to 38

Scale: it is used to determine number of digits to store decimal value within the precision.

The range of Scale is -84 to 127.

Scale is not mandatory.

The default value of scale is 0.

Ex:-

P>S NUMBER(3) $\Rightarrow$ 9 9 9

P>S NUMBER(3,2) $\Rightarrow$ 9 9 9

P>S NUMBER(5,3) $\Rightarrow$ 9 9 9 9 9

P>S NUMBER(8,1) $\Rightarrow$ 9 9 9 9 9 9 9 9

P=S NUMBER(4,4) $\Rightarrow$ 9 9 9 9

P>S NUMBER(8,0) $\Rightarrow$ 9 9 9 9 9 9 9 9

P<S NUMBER(3,5) $\Rightarrow$ 0 0 9 9 9

P<S NUMBER(8,12) $\Rightarrow$ 0 0 0 0 9 9 9 9 9 9 9 9

FORMAT: DD-WW-AA, 05, DD-WW-AAAA,

5/07/24

Large object

* Character large object:- It is used to store large amount of characters up to 4gb of size.

Syntax:- CLOB

* Binary large object:-

It is used to store binary values of Images, mp3, mp4, documents etc. up to 4gb of size.

Syntax:- BLOB

CONSTRAINTS:-

It is a rule which is assigned to a column to validate the data. It is known as constraints.

Types of Constraints:-

1. UNIQUE
2. NOT NULL
3. CHECK
4. PRIMARY KEY
5. FOREIGN KEY

NUMBER(10) VARCHAR(30) NUMBER(8,2) NUMBER(10)

EID	ENAME	SAL	PHNO
1	Neetu	35000	1234567890
2	Harsha	30000	1234567891
3	Varsha	-65000	1234567890
4	Poojitha	30000	1234567891
5	Sirce		
6		37000	12345

(sample check (length(phno)=10))			
NN	NN	NN	NN
UN			UN
EID	ENAME	SAL	PHNO
1	Neetu	35000	1234567890
2	Harsha	30000	1234567891
3	Varsha	-65000	1234567890
4	Poojitha	30000	1234567891
5	Sirce		
6		37000	12345

1. UNIQUE

It is a constraint which is not accepting duplicate/repeated values.

2. NOT NULL

It is a constraint which is assigned to a column which should not accept NULL.

3. CHECK

It is a constraint that provides extra validation with a condition, if the condition is satisfied then the value is accepted else rejected.

4. PRIMARY KEY

It is a constraint which is used to identify a record uniquely from the table.

Characteristics of Primary key

- We can have only 1 PK in a table
- PK cannot accept NULL
- PK cannot accept duplicate or repeated values.
- PK is always a combination of Unique & Not Null Constraint.
- It is not mandatory but highly recommended.

5. FOREIGN KEY

It is a constraint which is used to establish connection between the tables.

Characteristics of Foreign key

- We can have multiple FK in a table.
 - FK can accept NULL
 - FK can accept duplicate or repeated values
 - FK is not a combination of Unique & Not Null constraint.
 - FK is present in child table but actually belongs to parent Table.
 - FK is also known as Referential Integrity constraint.
 - For an attribute to become foreign key it must be a primary key
- in its table:

PK	NN	check	check	check	check
P.K	NN	NN	NN	NN	NN
EID	ENAME	SAL	PHNO	D.NO	CID
1	Smith	800	984...	10	1001
2	Allen	2000	684...	20	1002
3	Miller	3000	734...	30	1003
4	Scott	1500	984...	10	1004
5	King	5000	87783...		
6	Adams	3000	984...		

Parent

Customer

PK	CName	PHNO
1001	Dingri	123...
1002	Dingri	1234...
1003	Mangi	1237...

Parent

child

Difference b/w primary key and foreign key.

primary key	foreign key
→ unique record identification is possible/happen	→ It is not accept duplicate values
→ It does not accept duplicate values	→ It accepts NULL
→ It does not accept NULL	
→ Every table have only 1 pk	→ multiple foreign keys in a table can exist
→ Data can exist	
→ It is not mandatory	→ foreign key it must be a primary key.
→ Data highly recommended	→ It is also known as Referential Integrity Constraint

ERROR: ACCOUNT LOCK

Step 1:-

Username: system

Password: Tiger

Step 2:-

ALTER USER SCOTT →

↓

username

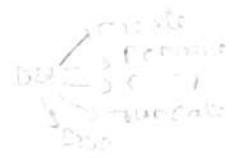
Account unlock;

Step 3:-

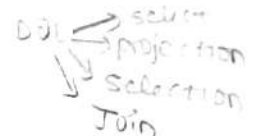
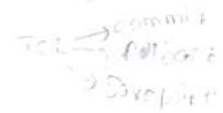
ALTER USER SCOTT

Identified by tiger;

Overview of SQL statements :-



1. Data Definition language (DDL)
2. Data Manipulation language (DML)
3. Transaction Control language (TCL)
4. Data control language (DCL)
5. Data Query language (DQL)



* Data Definition language :- This statement is used to construct/create a object (Table) and manipulate the structure of the object (Table)

Create table :-

It is used to constraints a object.

Name of the table : Movies

Number of the columns : 4

	M-ID	MName	rating	collection
Datatypes	VARCHAR(15)	VARCHAR(30)	NUMBER(1)	NUMBER(10)
Constraints	UNIQUE	NOT NULL	NOT NULL	NOT NULL
→ Check				
PRIMARY KEY	PK			Collection > 0

Syntax :-

create table tablename

(
 column_Name1 Datatype Constraint,
 column_Name2 Datatype constraint,
 column_Name3 Datatype Constraint,
 column_Name4 Datatype constraint
);

Ex :-

Create table movies

(
 M-ID varchar(15) primary key,
 Mname varchar(30) NOT NULL,
 rating Number(1) NOT NULL,
 collection Number(10) NOT NULL check (collection > 0)
);

check (collection > 0)

Name of the table: Teacher table

Number of the columns: 4

	TID	TName	Subject	Age
Datatype	Varchar(20)	Varchar(30)	Varchar(20)	Number(20)
Unique				
NOTNULL		NN	NN	NN
check				check (Age >= 20)
primary key	PK			

Create table Teacher

TID Varchar(20) primary key
TName Varchar(30) NOT NULL

Tsubject Varchar(20) NOT NULL,

Age Number(20) NOT NULL Check (Age >= 20)

CREATE TABLE Teacher (TID Varchar(20) PRIMARY KEY, TName Varchar(30) NOT NULL, Tsubject Varchar(20) NOT NULL, Age Number(20) NOT NULL CHECK (Age >= 20))

Create table with Foreign key

Name of the table: Department

Number of the columns: 3

D-ID	DName	LOC
(5)	Varchar(20)	Varchar(3)
PK	NN	NN

Department is parent table

D-ID Varchar(5) NOT NULL

Name of the table: Employees

Number of the columns: 5

E-ID	EName	Sal	Gender	DID
(5)	Varchar(40)	Number(8,2)	Varchar(8)	Number(5)
PK	NN	NN	NN	FK

child table

create table Department

(create table Employees)

```
(
  D-Id number(5) primary key,
  Dname varchar(20) NotNull,
  Loc varchar(30) Not Null
);
```

create table Employees.

```
(
  e-Id number(5) primary key,
  ename varchar(20) Not Null,
  Sal number(8,2) Not Null,
  Gender? varchar(20) Not Null,
  D-Id number(5) References Department (D-Id)
);
```

Rename

It is used to rename the table (object)

Syntax:-

Rename table-name to new-table-name;

Alter

Eg:- Rename movies to cinema;

Ex:- Alter table employees

Add email-id varchar(20) NotNull;

Syntax:-

Alter table table-name

Add column-name(size) Datatype constraint;

Drop

Ex:-

Syntax:-

Alter table table-name

Drop column col-name;

Alter table table employees

Drop column Gender;

Rename a column

Syntax:-

Alter table table-name

Rename column col-name to new.col-name;

11/07/24

* Modify the datatype :-

Syntax :-

Alter table tablename

modify col-name newdatatype (size);

Eg:- Alter table employees

modify emp_num varchar(10);

* Modify the NOT NULL constraint / NULL :-

Syntax :-

Alter table tablename

modify col-name datatype (size) NULL;

modify sal(8,2) NOT

NULL; modify sal number(8,2) NULL;

Eg:- Alter table employees

* Add a primary key :-

Syntax :-

Alter table tablename

Add primary key (col-name);

Eg:- Alter table tablename (employees)

Add primary key (emp-num);

* To drop a primary key :-

Syntax :-

Alter table tablename

Drop primary key;

Eg:- Alter table employees

Drop primary key;

* To check primary key :-

Syntax :- SET lines 1000 pages 1000;

Select * from user-Constraints;

* To ADD foreign key :-

Syntax :- Alter table tablename

Add foreign key (column-name) references parent-table-name (column-name);

* To Drop the foreign :-

Syntax :-

Alter table tablename

Drop constraint fk_ref_name;

Ex:- Alter table employees

Drop constraint SYS-C005410;

Name of the table :- course

No. of columns :- 3

Create table course

	course_id	course_name	department_id
DT	Varchar(6)	Varchar(20)	Varchar(6)
PK	PK		
NN	NN	NN	NN
Check			
unique			
FK			

(
course_id varchar(6) primary key,
course_name varchar(20) not null,
department_id varchar(6) not null,
);

Name of the table :- student

No. of columns :- 3

Create table student

	stu_id	stu_name	dept_id
DT	Varchar(6)	Varchar(20)	Varchar(6)
PK	PK		
NN	NN	NN	NN
Check			
unique			
FK			

(
student_id varchar(6) primary key,
student_name varchar(20) not null,
department_id varchar(6) not null,
);

Name of the table :- enrollment

No. of columns :- 3

Create table enrollment

	stu_id	course_id	enrollment_date
DT	Varchar(6)	Varchar(20)	Date
PK			PK
NN	NN	NN	NN
Check			
unique			
FK	FK	FK	

(
student_id varchar(6) references
student(student_id),
course_id varchar(20) references
course(course_id),
enrollment_date date primary key
);

Truncate :-

It is used to remove all the records from the table permanently

Syntax :-

Truncate table table_name;

To check database

Select *
from tab;

Drop :-

It is used to remove the table from the database

Syntax :- Drop table table_name

ex:- Drop table employees

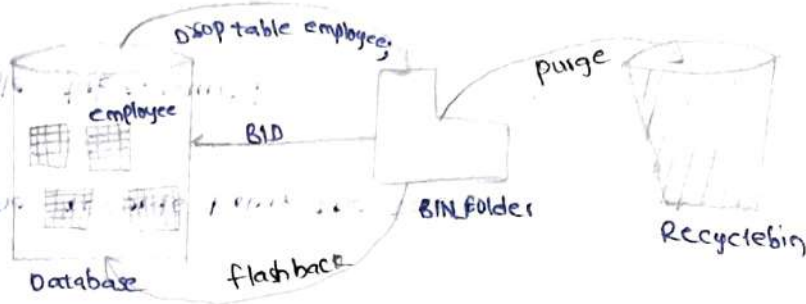
To recover the table:- Syntax:- flashback table tablename

To drop before

ex:- flashback table employees To before drop;
to before drop;

To delete the table from BIN folder

Syntax:- purge table table-name;



→ DDL is autocommit statement

Data Manipulation Language (DML)

It is used to manipulate the object by performing insertion, updating and deletion.

1. INSERT:-

It is used to insert/create records in the table

Syntax:- insert into tablename
values (v1, v2, v3 ...);

Ex:- insert into Department values (10, 'datascience', 'Gunter');

2. Update:-

It is used to modify an existing value.

Syntax:- Update table-name
set colname = value, colname = value, ...
[where stmt];

Ex:-
update emp
set sal = 2000

Assignment

insert into student values (15, 'saritha', 25);
insert into student values (25, 'seetha', 20);
insert into student values (35, 'geetha', 20);

Studentid	name	Departmentid
15	saritha	25
25	Seetha	20
35	geetha	20

Insert into course values (100, 'bse', 20);

Insert into course values (110, 'bcom', 30);

Insert into course values (120, 'btech', 20);

Insert into enrollment values (15, 100, '12/June/2024');

Insert into enrollment values (35, 110, '15/June/2024');

Insert into enrollment values (25, 120, '13/June/2024');

Course-id	Course-name	Department-id
100	bse	20
110	bcom	30
120	btech	20

Student-id	Course-id	Enrollment date
15	100	12/June/2024
35	110	15/June/2024
25	120	13/June/2024

13/07/24

3. Delete:-

It is used to remove a particular record from the table

Syntax:- delete from table

[Where stmt];

ex:- delete from student

where student_id = 45;

Data Transaction

Control Language

→ It is used to save the transactions

Commit -

Used to save the transactions

Syntax:- commit;

Rollback -

Used to rollback / Get back to the latest commit point

Syntax:-

rollback;

Savepoint -

It is used to mark the transaction

Syntax:- `Savepoint: savepoint-name;`

→ savepoint we can use along with rollback

Data Control Language:-

`Rollback to savepoint-name;`

It is used to grant the permission or take back the permission.

Grant:-

It is used to grant permission to the user

Syntax:-

`Grant SQL-statements on table-name`

`TO user-name;`

Ex:- `Grant select on emp`

`to hr;`

Revoke:-

It is used to take back the permission from the table

Syntax:-

`Revoke SQL-statements on table-name`

`from user-name;`



Given the permission
Grant

Take back the permission
Revoke

→ `Grant select on emp`

`to hr;`

→ `Revoke select on emp`

`from hr;`

`select * from emp;`
does not exist

→ `select * from scott.emp;`

After Revoke

`select * from scott.emp;`

table does not exist

Data Query language:-

'DQL is used to retrieve the data from the database'.

It had 4 statements.

1. select,
2. projection
3. selection
4. join

1. select: 'It is used to retrieve the data from the table and display it.'

2. projection: 'It is a process of retrieving the data by selecting only the columns is known as projection'.
In projection all the records/values present in a particular column are by default selected.

3. Selection:-

'It is a process of retrieving the data by selecting both the columns and rows known as selection.'

4. join:-

'It is a process of retrieving the data from multiple tables simultaneously is known as join'.

projection:- 'It is a process of retrieving the data by selecting only the columns is known as projection'.
In projection all the records/values present in a particular column are by default selected.

Syntax:-

Select, #/[DISTINCT] column-name / expression [Alias]
from table-name;

Ex:- WAGTD EName of all the employee.

Select Ename

from emp;

o/p of a select clause

it will go to table emp and find employee info have data from table and display emp



Database

o/p of from clause

EName
Smith
Allen
Scott
Miller
King
Blake

EmpNo	EName	Sal	DeptNo
1	SMITH	800	10
2	ALLEN	2500	20
3	SCOTT	1500	30
4	MILLER	3000	20
5	KING	5000	10
6	BLAKE	2500	30

under execution

It is a process of retrieving the data by selecting

1. WAGTD name and salary of all the employees.

Select Ename, Sal

from emp;

Ename	Sal
Smith	800
Allen	1600
WARD	1250
Jones	2975
Martin	1250
Blake	2850
CLARK	2450
Scott	3000
KING	5000
Turner	1100

Adams	1100
James	950
FORD	3000
Miller	1300

2. Wqtd details of all the employees.

```
select *  
from emp;
```

3. Wqtd dname of the employees

```
select dname  
from department;
```

4. Wqtd ename and hiredate of all the employees.

```
select ename, hiredate  
from emp;
```

5. Wqtd ename, empno, salary, mgr, hiredate, deptno and job of all the employees

```
select ename, empno, sal, mgr, hiredate, deptno, job  
from emp;
```

Assignment :-

1. Wqtd display all the details from the employee table.

```
select *  
from emp;
```

2. Wqtd names of all the employees.

```
select ename  
from emp;
```

3. Wqtd name and salary given to all the employees.

```
select ename, sal  
from emp;
```

4. Wqtd name and commission given to all the employees.

```
select ename, comm  
from emp;
```

5. WABTD employee ID and department number of all the employees in emp table.

Select empno, deptno

from emp;

6. WABTD ename and hiredate of all the employees.

Select ename, hiredate

from emp;

7. WABTD ename and designation of all the employees.

Select ename, job

from emp;

8. WABTD ename, job and salary given all the employees.

Select ename, job, sal

from emp;

9. WABTD Dnames present in Department table.

Select Dname

from department;

10. WABTD Dname and location present in Dept table.

Select Dname, loc

from department;

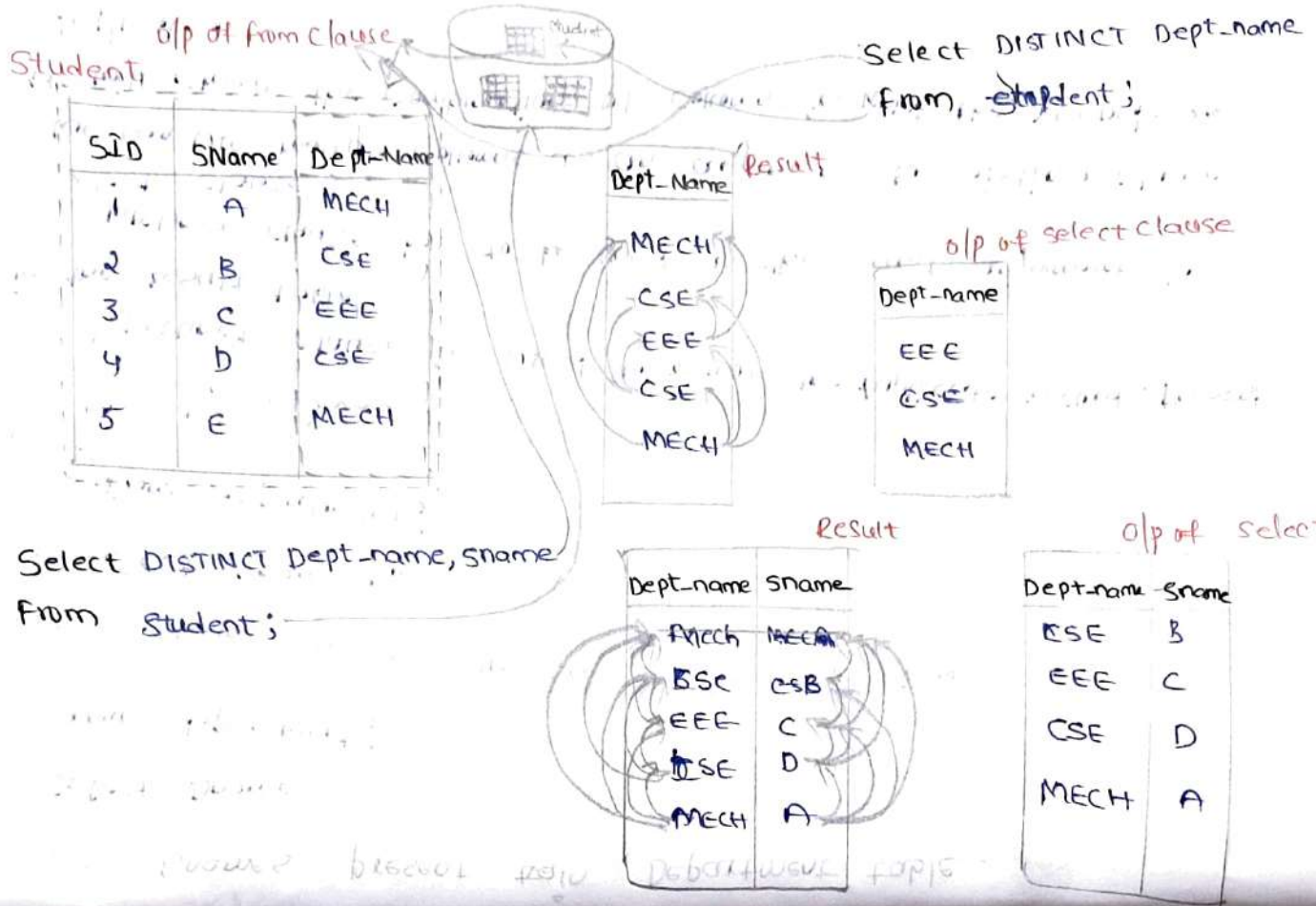
17/10/24

DISTINCT CLAUSE (Different / Separate)

'Distinct clause is used to remove the duplicate records present in the result table.'

Distinct clause has to be used as the first argument to Select clause.

We can use multiple columns as an argument to distinct clause, it will remove the combination of columns in which the records are duplicated.



Expression:-

- Any statement which gives result is known as Expression
- It is a combination of operand and operator

EX:-

$$2 + 2 = 4$$

↑ operator

↑ operands

Operands:-

It has two types

1. columnname

2. literals

Literals:-

It has three types

1. character literal

2. Number literal

3. date literal

Note:- character and date enclosed with single quote.

19/07/24

1. Write the SQL query to find the annual salary of the employees.

```
Select ename, sal * 12  
from emp;
```

2. Write the SQL query to find the name of the employees along with hike the 10% of all the employees salary.

```
Select ename, sal + sal * 10/100  
from emp;
```

3. Write the SQL query to find the name of the employees along with hike the 10% of all the employee annual salary.

```
Select ename, sal * 12 + sal * 12 * 10/100  
from emp;
```

4. Write the SQL query to find the details of the employees along with annual salary of all the employees.

```
Select ename, deptno, empno, mgr, job, hiredate, sal * 12  
from emp;
```

→ Select emp.*, sal * 12

```
from emp;
```

5. Select the details of employee along with hike with 25% of annual salary.

```
Select emp.*, sal * 12 + sal * 12 * 25/100  
from emp;
```

ALIAS

"It is an alternate name given to a column or an expression in the result table."

Alias names must always be a single string which is separated by underscore or enclosed within double quotes ""

```
EX:- SELECT sal * 12 AS "Annual salary"  
from emp;
```

```
OR: SELECT sal * 12 AS annual_salary
```

OR: SELECT sal * 12 AS annual_salary

we can assign alias name with or without using 'As' keyword

ex: select sal * 12 as annual-salary
from emp;

→ select sal * 12 annual-salary
from emp;

Assignment

1. WABTD name of the employees with their annual salary

Select ename, sal * 12 "Annual salary"
from emp;

2. WABTD the ename and job for all the employee with their half term salary

Select ename, job, sal * 6 "half-term-sal"
from emp;

3. WABTD all the details of the employees along with an annual bonus of 2000

Select emp.*, sal * 12 + 2000
from emp;

4. WABTD all the details of the employees along with annual salary.

Select emp.*, sal * 12 as annual-sal
from emp;

5. WABTD name salary and salary with a hike of 10%

Select ename emp.ename, emp.sal, sal + sal * 10/100
from emp;

6. WABTD name and salary with deduction of 25%

Select ename, sal - sal * 25/100
from emp;

7. WABTD name and salary with monthly hike of 50%

Select ename, sal + 50
from emp;

8. wantd name and Annual salary with deduction of 10%
 Select ename, sal*12 - sal*12*10/100
 from emp;

9. wantd total salary given to each employee (sal+comm).
 Select emp.sal, sal+comm as "Total Salary"
 from emp;

10. wantd name and designation along with 100 penalty in salary.
 Select ename, job, sal+100
 from emp;

20/07/24

Selection

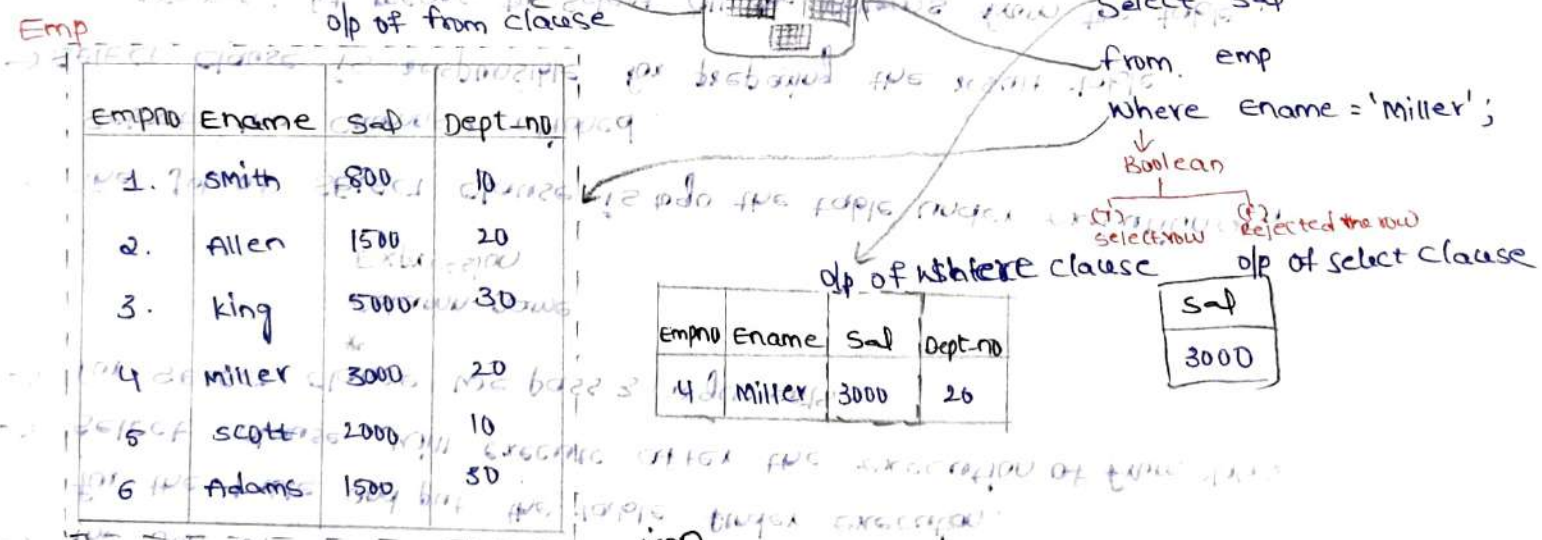
The process of retrieving the data by selecting only the both the column and row is known as Selection

Syntax

Select #/DISTINCT column-name/Expression [ALIAS]
 from Table-name
 where filter-condition > ;

Order of execution

1. from clause
2. where clause (row by row)
3. select clause



under execution

for the where clause we can filter rows as an condition

from clause starts the execution.

- For from clause we can pass table-name as an argument.
- The job of from clause is to go to the database and search for the table and put the table under execution.
- select clause will execute after the execution of from clause
- For select clause we pass 3 arguments:

*
Column-name
Expression

- The job of select clause is to go to the table under execution and select the columns mentioned.

- select clause is responsible for preparing the result table.

Asterisk: It means to select all the columns from the table.

Semicolon: It means end of the query.

Where clause:

Note:-

1. WHERE clause executes after from clause
2. WHERE clause executes row by row
3. We can write filter condition in where clause, we can also write multiple filter condition in where clause with the help of logical operators.

① Write a query to find the details of the employees who are not working in deptno 10.

```
SELECT ename, sal, deptno  
FROM emp  
WHERE deptno != 10;
```

② Write a query to find the details of the employees whose salary is more than 2500.

```
SELECT *  
FROM emp  
WHERE sal > 2500;
```

3. want details of the employees who are hiredate after 1981

Select *

from emp

where hiredate > '31/Dec/1981';

4. want details of the allen.

Select *

from emp

where ename = 'ALLEN';

Assignment

1. want the annual salary of the employees whose name is Smith

Select sal*12

from emp

where ename = 'SMITH';

2. want name of the employees working as clerk.

Select ename

from emp

where job = 'CLERK';

3. want salary of the employees who are working as salesman

SELECT sal

from emp

where job = 'SALESMAN';

4. want details of the employees who earns more than 2000

Select *

from emp

where sal > 2000;

5. want details of the emp whose name is Jones.

Select *

from emp

where ename = 'JONES';

6. want details of the emp who has hired after 01-Jan-81

Select *

from emp

where hiredate > '01-Jan-81';

7. want name and sal along with his annual salary if the annual salary is more than 12000

Select ename, emp.ename, emp.sal, sal*12

from emp

where sal*12 > 12000;

8. Wqytd empno: of the employees who are working in dept 30
 select empno
 from emp
 where deptno = 30;
9. Wqytd ename and hiredate if they are hired before 1981
 select ename, hiredate
 from emp
 where hiredate < '01-Jan-81';
10. Wqytd details of the employees working as manager.
 select *
 from emp
 where job = 'MANAGER';
11. Wqytd name and salary given to an employee if the employee earns a commission of rupees 1400
 select ename, sal
 from emp
 where comm > 1400;
12. Wqytd details of the employees having commission more than salary.
 select *
 from emp
 where comm > sal;
13. Wqytd empno of employees hired before the year 87.
 select empno
 from emp
 where hiredate < '01-Jan-87';
14. Wqytd details of employees working as an Analyst
 select *
 from emp
 where job = 'ANALYST';
15. Wqytd details of emps earning more than 2000 rupees per month
 select *
 from emp
 where sal > 2000;

Operators in SQL

22/07/24

1. Arithmetic operators :- (+, -, *, /)
2. Concatenation operators :- (||)
3. Comparison operator :- (=, != or <>)
4. Relational operator :- (>, <, >=, <=)
5. Logical operator :- (AND, OR, NOT)

6. Special operators :-

1. IN
 2. NOT IN
 3. BETWEEN
 4. NOT BETWEEN
 5. IS
 6. IS NOT
 7. LIKE
 8. NOT LIKE
- } pattern matching.

7. Subquery operators :-

1. ALL
2. ANY
3. EXISTS
4. NOT EXISTS

8. Logical operators :-

1. AND
2. OR
3. NOT

Logical operator such as AND, OR and must be used between the conditions.

* AND Operator :-

AND op is used whenever we've to satisfy all the conditions.

* OR Operator :-

OR operator is used whenever we've to satisfy any one of the conditions.

* NOT Operator :-

NOT operator is a unary operator [it can have only 1 operand] It is used to negate the output. [inverse]

1.

Wanted name of the employees whose salary is more than 2000 and less than 4000.

```
SELECT ename, sal  
FROM emp  
WHERE sal > 2000 and sal < 4000;
```

2. want details of the employees who are hired on after 81 and before 87

Select *

from emp

where hiredate > '01-Dec-81' and hiredate < '01-Jan-87';

3. want details of the employees who are working on deptno 20 or 30.

Select *

from emp

where deptno = 20 OR deptno = 30

4. want details of the employees who are working as manager or salesman.

Select *

from emp

where job = 'MANAGER' or job = 'SALESMAN';

5. want details of the employees along with annual salary if annual salary more than 8000 and less than 12000

Select emp.*, sal*12

from emp

where sal*12 > 8000 and sal*12 < 12000;

6. want details of the employees who are working in deptno 10, 20 and salary less 3000.

Select *

from emp

where (deptno = 10 OR deptno = 20) and sal < 3000;

Assignment

1. Write details of the employees working as clerk and earning less than 1500
select *
from emp
where job='CLERK' and sal<1500;
2. Write name and hiredate of the employees working as manager in dept 30
select ename, hiredate
from emp
where job='MANAGER' and deptno=30;
3. Write details of the emp along with annual salary if they are working in dept 30 as salesman and their annual salary has to be greater than 14000.
select emp.*, sal*12
from emp
where deptno=30 and job='SALESMAN' and sal*12>14000;
4. Write all the details of the employee working in dept 30 OR AS ANALYST.
select *
from emp
where deptno=30 or job='ANALYST';
5. Write names of the employees who is salary is less than 1100 and their designation is clerk.
select ename
from emp
where sal<1100 and job='CLERK';
6. Write name and sal, Annual sal and deptno if deptno is 20 earning more than 1100 and annual salary exceeds 12000.
select ename, emp.sal, sal*12, deptno
from emp
where deptno=20 and sal>1100 and sal*12>12000;

7. Writd empno and names of the employees working as manager and in deptno 20.

```
select empno, ename  
from emp  
where job = 'MANAGER' and deptno = 20;
```

8. Writd details of employees working in dept 20 OR 30

```
select *  
from emp  
where deptno = 20 OR deptno = 30;
```

9. Writd details of employees working as analyst in Dept 10

```
select *  
from emp  
where job = 'ANALYST' and deptno = 10;
```

10. Writd details of employees working as president with salary of rupees 4000

```
select *  
from emp  
where job = 'PRESIDENT' and sal = 4000;
```

11. Writd names and deptno, job of employees working as clerk in dept 10 OR 20.

```
select ename, deptno, job  
from emp  
where job = 'CLERK' and (deptno = 10 OR deptno = 20);
```

12. Writd details of employees working as clerk OR manager in dept 10.

```
select *  
from emp  
where (job = 'CLERK' OR job = 'MANAGER') and deptno = 10;
```

13. Writd names of employees working in Dept 10, 20, 30, 40

```
select ename  
from emp  
where deptno = 10 OR deptno = 20 OR deptno = 30 OR deptno = 40;
```

14. Writd details of employees with empno 7902, 7839

```
select *  
from emp  
where empno = 7902 OR empno = 7839
```

15. Write details of employees working as manager or Salesman or clerk

```
select *  
from emp  
where job = 'MANAGER' OR job = 'SALESMAN' OR job = 'CLERK';
```

16. Write names of employees hired after 81 and before 87

```
select ename  
from emp  
where hiredate > '31-DEC-81' AND hiredate < '01-JAN-87';
```

17. Write details of employees earning more than 1250 but less than 3000

```
select *  
from emp  
where sal > 1250 and sal < 3000;
```

18. Write names of employees hired after 81 into deptno 10 OR 30.

```
select ename  
from emp  
where hiredate > '31-DEC-81' AND (deptno = 10 OR deptno = 30);
```

19. Write names of employees along with Annual salary for the employees working as manager or clerk into deptno 10 OR 30

```
select ename, sal * 12  
from emp  
where (job = 'MANAGER' OR job = 'CLERK') and (deptno = 10 OR deptno = 30);
```

20. Write all the details along with annual salary if sal is between 1000 and 4000 annual salary more than 15000.

```
select emp.*, sal * 12  
from emp  
where sal > 1000 and sal < 4000 and sal * 12 > 15000;
```

23/07/21 Special Operators

IN operator:-

It is a multivalued operator which can accept multiple values at the RHS.

Syntax:- Column_name/Expression IN (V₁, V₂, V₃... V_n);

Ex:- Wagt name of the employees who are working in dept 10, 20, 30, 40

Select ename

from emp

where deptno = 10 OR deptno = 20 OR deptno = 30 OR deptno = 40;

Using IN operator

Select ename

from emp

where deptno in (10, 20, 30, 40);

deptno = 10, 20, 30, 40

↓
equal operator accepts

single values at RHS

Not in:-

It is similar to IN operator, instead of selecting the values

it rejects the values

Syntax:-

Column_name/Expression NOT IN (V₁, V₂, V₃... V_n);

Ex:- Wagt name of the employee except who are working deptno 10, 20, 40

Select ename

from emp

where deptno not in (10, 20, 40);

BETWEEN:-

It is used whenever we have range of values.

[start value, and end value]

Syntax:-

Column_name/Expression Between low-range And high-range;

Ex. Wagt ename of the employees who are earning salary 1000 to 3000

Select ename

from emp

where sal between 1000 and 3000;

Note:-

We cannot interchange the range

It works including the range specified.

Not between:- It is opposite of between operator.

Syntax:- Column-name Not Between lower-range and Higher-range;

Ex:- Wrt'd name of the employee except who are earning salary 1000 to 3000.

Select ename

from emp

where sal Not between 1000 and 3000;

2. Wrt'd name of the employee who are earning salary more than 1000 and less than 3000

Select ename

from emp

where sal between 1001 and 2999;

2. Wrt'd name of the employee who are earning salary between 1000 to 3000.

Select ename

from emp

where sal between 1001 and 2999;

3. Wrt'd name of the employee who are hire after 01-Jan-1981 and before 31-Dec-1988

Select ename

from emp

where hiredate Between '01-Jan-1981' and '31-Dec-1988';

4. Wrt'd name of the employee who are hiredate after 1980 and before 1989.

Select ename

from emp

where hiredate between '01-Jan-1981' and '31-Dec-1988';

5. Wrt'd name of the employees who are hired between 80 TO 90

Select ename

from emp

where hiredate between '01-Jan-1981' and '31-Dec-1988';

24/07/24

IS:- IS operator is used to compare with NULL

Syntax:- column_name/Exp IS NULL;

IS not:- IS not operator is used to compare with not NULL

Syntax:- column_name/Exp IS NOT NULL;

1. waqtd details of the employee who are not getting Comm

```
Select *
from emp
where comm is null;
```

2. waqtd details of the employee who are getting Comm.

```
Select ename
from emp
where comm is not null;
```

Like:-

It is used for pattern matching

To achieve pattern matching we use special characters

(i) Percentile (%)

(ii) underscore (_)

Syntax:- column_name like 'pattern-to-match';

1. waqtd name of the employees whose name start with char 'A'

```
Select ename
from emp
where ename like 'A%';
```

2. waqtd name of the employees whose name starts with char 'A' and end with char 'E'

```
Select ename
from emp
where ename like 'A%E';
```

3. List name of the employees whose name ends with 'L'

```
SELECT ename
FROM emp
WHERE ename LIKE '%L';
```

4. List name of the employees whose name starts with 'AEIOU'

```
SELECT ename
FROM emp
WHERE ename LIKE 'A%' OR
      ename LIKE 'E%' OR
      ename LIKE 'I%' OR
      ename LIKE 'O%' OR
      ename LIKE 'U%';
```

NOT LIKE

It is similar to like operator instead of selecting values it rejects

Syntax: `column-name exp NOT LIKE 'pattern-match';`

Assignment:-

1. List all the employees whose commission is NULL

```
SELECT ename
FROM emp
WHERE comm IS NULL;
```

2. List all the employees who don't have a reporting manager

```
SELECT ename, emp.*
FROM emp
WHERE mgr IS NULL;
```

3. List all the salesmen in dept 30

```
SELECT ename
FROM emp
WHERE job = 'SALESMAN' and deptno = 30
```

4. List all the salesmen in dept 30 and having salary greater than 1500

```
SELECT ename
FROM emp
WHERE job = 'SALESMAN' and deptno = 30 and sal > 1500;
```

5. List all the employees whose name starts with 'S' OR 'A'

```
select ename  
from emp  
where ename like 'S%' OR ename like 'A%';
```

6. List all the employees except those who are working in dept 20 & 10

```
select ename  
from emp  
where deptno not in (10, 20);
```

7. List the employees whose name does not start with 'S'

```
select ename  
from emp  
where ename not like 'S%';
```

8. List all the employees who are having reporting managers in dept 10

```
select *  
from emp  
where mgr is not null and deptno = 10;
```

9. List all the employees whose commission is NULL and working as clerk

```
select ename  
from emp  
where comm is null and job = 'CLERK';
```

10. List all the employees who don't have a reporting manager in dept 10 OR 30

```
select *  
from emp  
where mgr is null and (deptno = 10 OR deptno = 30);
```

11. List all the salesmen in dept 30 with sal more than 2450

```
select ename  
from emp  
where job = 'SALESMAN' and deptno = 30 and sal > 2450;
```

12. List all the analyst in dept 20 and having salary greater than 2500

```
select ename  
from emp  
where job = 'ANALYST' and deptno = 20 and sal > 2500;
```

13. List all the employees whose name starts with 'M' OR 'J'

Select ename

from emp

where ename

14. List all the employees with annual salary except those who are working in dept 30

Select ename, sal * 12

from emp

where deptno not in 30;

15. List the employees whose name does not end with 'ES' OR 'R'

Select ename

from emp

where ename not like '%ES' OR ename not like '%R';

16. List all the employees who are having reporting managers in dept 10 along with 10% hike in salary.

Select emp.*, sal + sal * 10/100

from emp

where mgr is not null and deptno=10; and sal + sal * 10/100;

17. Display all the employee who are Salesmen's having 'E' as the last but one character in ename but salary having exactly

4 character

Select *

from emp

where job = 'SALESMAN' and ename like '%E_' and

sal like '----';

18. Display all the employee who are joined after year 81.

Select *

from emp

where hiredate > 31-Dec-81;

19. Display all the employee who are joined in feb

Select *

from emp

where hiredate like 'FEB--';

20. List the employees who are not working as managers and clerk in dept 10 and 20 with salary in the range of 1000 to 3000

Select *

from emp

where job != 'MANAGER' and job != 'CLERK' and sal between 1000 and 3000 and deptno not in (10, 20);

Select *

from emp

where job not in ('MANAGER', 'CLERK') and deptno not in (10, 20) and

sal between 1000 and 3000;

25/04/24 :-

functions

It is set of code or list of instructions which are used to perform a specific task.

There are 3 main components of function.

1. Function name

2. Number of arguments (No. of inputs)

3. Return type.

User defined function

↓
in PL/SQL

Types of functions:

function



In Built functions

↓

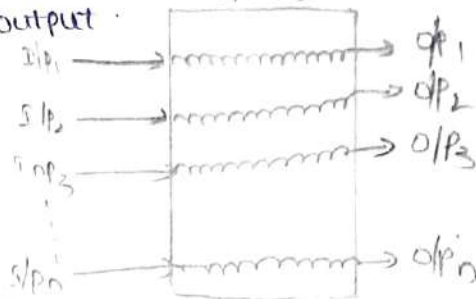
Types of functions in SQL:-
Single row function / Multi row function / Aggregate functions / group functions.

1. Single row function / scalar function.

2. Multi row function / aggregate function / group functions.

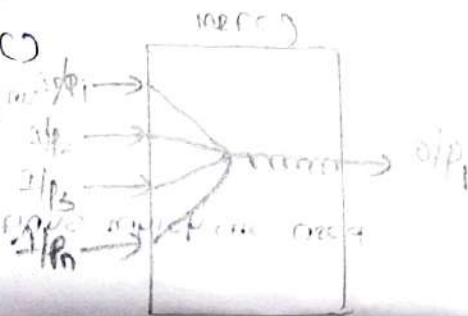
1. Single row function / Scalar function:-

- Single row function execute row by row
- Single row function take one input at a time process and execution
- If we pass 'N' number of input to single row function
it will always return 'N' number of output



2. Multi row function:-

- Multi row functions executes group
- It takes all the inputs at one shot and then executes and provides a single output
- If we pass 'N' number of inputs to a MRFC
it returns '1' output or single output



26/07/24

function 2

Example for single row function

Length:- select length(ename)
from emp;

Example for multi-row function

Max:- select max(sal)
from emp;

List of Multi-row / Aggregate function:-

- max() → It is used to get maximum value
- min() → It is used to get minimum value
- avg() → It is used to get average value
- sum() → It is used to get total sum value
- count() → It is used to get number of values

Note:-

- Multi-row function will not accept more than 1 arguments / we can't accept more than 1 colname as argument
- Along with multi-row function, we can't pass any other col-name / Expression

→ Multi-row function cannot accept where clause.

→ Multi-row function "avoid null value"

→ Count function only accepts the hashtick (*) as argument

1. Wqtd maximum salary of manager.

```
select salmax(sal)
from emp
where job = 'MANAGER';
```

2. Wqtd maximum salary of deptno 10

```
select max(sal)
from emp
where deptno = 10;
```

3. Wqtd number of employee working in deptno 30

```
select count(*)
from emp
where deptno = 30;
```

Example for single row function

4. Wqtd number of employee working in each dept.

```
select count(ename)
from emp
where
group by (deptno);
```

5. Wqtd number of different salaries present in emp.

```
select count(distinct sal)
from emp;
```

6. Wqtd number of records present into the emp

```
select count(*)
from emp;
```

Wqtd number of employees in deptno 10

Assignment

1. Wqytd number of employees getting salary less than 2000 in deptno 10

```
select count(sal)
from emp
where sal < 2000 and deptno = 10;
```

2. Wqytd Total salary needed to pay employees working as clerk

```
select sum(sal)
from emp
where job = 'CLERK';
```

3. Wqytd average salary needed to pay All employees

```
select avg(sal)
from emp;
```

4. Wqytd number of employees having 'A' as their first character

```
select count(ename)
from emp
where ename like 'A%';
```

5. Wqytd number of employees working as clerk or manager

```
select count(ename)
from emp
where job = 'MANAGER' or job = 'CLERK';
```

6. Wqytd Total salary needed to pay employees hired in feb.

```
select sum(sal)
from emp
where hiredate like '%FEB%';
```

7. Wqytd number of employees reporting to 7839 (MGR)

```
select count(ename)
from emp
where mgr = 7839;
```

8. Wqytd number of employees getting comm in Deptno 30.

```
select count(comm)
from emp
where deptno = 30;
```

9. wantd Avg sal, total sal, number of empls and maximum salary given to employees working as president.

```
select avg(sal), sum(sal), count(sal), max(sal)
from emp
where job='PRESIDENT';
```

10. wantd number of employees having 'A' in their names.

```
select count(ename)
from emp
where ename like '%A%';
```

11. wantd number of emps and total salary needed to pay the employees who have 2 consecutive L's in their names

```
select count(ename), sum(sal)
from emp
where ename like '%LL%';
```

12. wantd number of Departments present in employee Table.

```
select count(Distinct deptno)
from emp;
```

13. wantd number of employees having character 'z' in their names.

```
select count(ename)
from emp
where ename like '%z%';
```

14. wantd number of employees having '\$' in their names.

```
select count(ename)
from emp
where ename like '%$%';
```

15. wantd total salary given to employees working as clerk in dept 30

```
select sum(sal)
from emp
where job='CLERK' and deptno=30;
```

16. Wqtd nummaximum salary given to the employees working as Analyst

Select max(sal)

from emp

where job='ANALYST';

17. Wqtd number of distinct salaries present in employees Table

Select count(distinct sal)

from emp;

18. Wqtd number of jobs present in employee table

Select count(distinct job)

from emp;

where job

19. Wqtd Avg salary given to the Clerk.

Select Avg(sal)

from emp;

where job='CLERK';

Select count(distinct deptno)

20. Wqtd minimum salary given to the employees who work in dept 10 as manager or a Clerk.

Select min(sal)

from emp;

where deptno=10 and (job='MANAGER' or job='CLERK');

29/07/24

1. Wqtd number of employees working in deptno 20

Select count(*)

from emp

where deptno=20;

2. Wqtd number of employees working in deptno 30

Select count(*)

from emp

where deptno=30;

Group by clause -> Group by clause is used to group the records/rows/tuple
 -> Group by clause executes row by row
 -> After the execution of group by clause we get groups.
 1. way to no. of employees

Empno	ename	deptno
1	Smith	10
2	Allen	20
3	Miller	30
4	Scott	20
5	King	10
6	Adams	30
7	Blake	20

-> Therefore any clause that executes after group by must execute group by group.

working in each dept

Select count(*), deptno
 From emp
 Group by deptno;

Syntax:-

op of from clause

Syntax:-

Select Group by expression / Group function
 from emp
 (where <filter condition>)

Group by column-name / Expression

o/p of group by clause

10	1-Smith 10	2
	5-King 10	
30	3-Miller 30	2
	6-Adams 30	
20	2-Allen 20	3
	4-Scott 20	
	7-Blake 20	

-> The column-name or expression used for grouping can be used in select clause.

-> Group by clause can be used without using where clause.

Order of execution

1. from clause

2. where (if used) (row by row)

3. group by (row by row)

4. select (Group by)

Assignment

1. way to number of employees working in each department except president.

Select count(*) from emp

where job not in ('PRESIDENT')

Group by deptno;

2. wanted total salary needed to pay all the employees in each

```
Job select sum(sal)
from emp
Group by (Job);
```

3. wanted number of employees working as manager in each department

```
select count(*)
from emp
where job = 'MANAGER'
Group by (deptno);
```

4. wanted avg salary needed to pay all the employees in each department excluding the employees of deptno 20

```
select avg(sal)
from emp
where deptno != 20
Group by (deptno);
```

5. wanted number of employees having character 'A' in their names in each job

```
select count(*)
from emp
where ename like '%A%'
Group by (Job);
```

6. wanted number of employees and avg salary needed to pay the employees whose salary is greater than 2000 in each dept.

```
select count(ename), avg(sal)
from emp
where sal > 2000
Group by (deptno);
```

7. wanted total salary needed to pay and number of salesmen in each dept.

```
select sum(sal), count(ename)
from emp
where job = 'SALESMAN'
Group by (deptno);
```

8. wanted number of employees with their maximum salaries in each job

```
select count(*), max(sal)
from emp
Group by (deptno);
```

9. wantd maximum salaries given to an employee working in each dept.

```
SELECT max(sal)
FROM emp
GROUP BY deptno;
```

10. wantd number of times the salaries present in employee table.

```
SELECT count(sal)
FROM emp
GROUP BY (sal);
```

30/07/24.

Having Clause

Having clause is used to filter the groups.

→ Having clause is used to filter the groups.

→ Having clause executes group by group.

→ Having clause we can use $max()$

→ Having clause executes after group by clause.

Syntax:-

SELECT group-by expression/group function.

FROM table-name.

[WHERE <filter-condition>]

GROUP BY column-name/Expression

HAVING <group-filter-condition>;

Order of execution:

1. FROM
2. WHERE (if used) (row-by-row)
3. GROUP BY (if used) (row-by-row)
4. HAVING (if used) (Group-by-group)
5. SELECT

o/p of from clause

Empno	ename	sal	deptno
1	ALLEN	800	10
2	SMITH	3000	20
3	MILLER	1500	30
4	SCOTT	2500	20
5	KING	5000	10
6	ADAMS	1500	30
7	BLAKE	3000	20

under execution.

o/p of group by clause

1	ALLEN	800	10
5	KING	5000	10

2	SMITH	3000	20
4	SCOTT	2500	20
7	BLAKE	3000	20

3	MILLER	1500	30
6	ADAMS	1500	30

Wagtd no. of emp working in each dept if there more than 2 emp working in it.

Select count(*), deptno

from emp
group by deptno,

having count(*) > 2;

clause
o/p of having

2	SMITH	3000	20
4	SCOTT	2500	20
7	BLAKE	3000	20

clause
o/p of select

count(*)	deptno
3	20

Assignment :-

1. Wagtd dno and number of emp working in each dept if there are atleast 2 clerks in each dept

Select count(*), deptno

from emp
where job = 'CLERK'
group by deptno

having count(*) >= 2;

2. Wagtd dno and total salary needed to pay all emp in each dept if there are atleast 4 emp in each dept.

Select sum(sal), deptno

from emp
where
Group by deptno

having count(ename) >= 4;

3. Wqytd number of emp earning sal more than 1200 in each job and the total sal needed to pay emp of each job must exceeds 3800.

```
select count(sal), job
from emp
where sal > 1200
group by job
having sum(sal) > 3800;
```

4. Wqytd deptno and number of emp working only if there are 2 emp working in each dept as manager.

```
select deptno, count(*)
from emp
where job = 'MANAGER'
group by deptno
```

```
having count(*) >= 2;
```

5. Wqytd job and max sal of emp in each job if the max sal

exceeds 2600.

```
select job, max(sal)
from emp
where
group by job
```

```
having max(sal) > 2600;
```

6. Wqytd the salaries which are repeated in emp table

```
select sal
```

```
from emp
```

```
group by sal
```

```
having count(sal) > 1
```

7. Wqytd the hiredate which are duplicated in emp table

```
select hiredate
```

```
from emp
```

```
group by hiredate
```

```
having count(*) > 1;
```

8. wantd avg salary of each dept if avg sal is less than 3000

```
select avg(sal)
```

```
from emp
```

```
group by deptno
```

```
having avg(sal) < 3000;
```

9. wantd deptno if there are atleast 3 emp in each dept whose name has char 'A' or 'S'

```
select deptno
```

```
from emp  
where ename like '%A%' or ename like '%S%'  
group by deptno
```

```
having count(*) > 3;
```

10. wantd min and max salaries of each job if min sal is more than 1000 and max sal is less than 5000.

```
select min(sal), max(sal)
```

```
from emp
```

```
group by job having min(sal) > 1000 and max(sal) < 5000;
```

11. Difference between where clause and having clause.

Where Clause

Having clause

1. filters row by row

1. Having clause executes

group by group

2. where clause exactly

2. Having clause

after from clause

exactly after group

by clause.

3. where clause is

3. Having clause is used

used to filter rows

to filter groups of rows

before any grouping

after the group by clause has

been performed.

been applied.

11/08/24

1. Wq:td name of the employee who are earning same salary.

```
Select ename, sal
from emp
where sal in (select sal
from emp
group by sal
having count(sal) > 1);
```

SUB Query:-

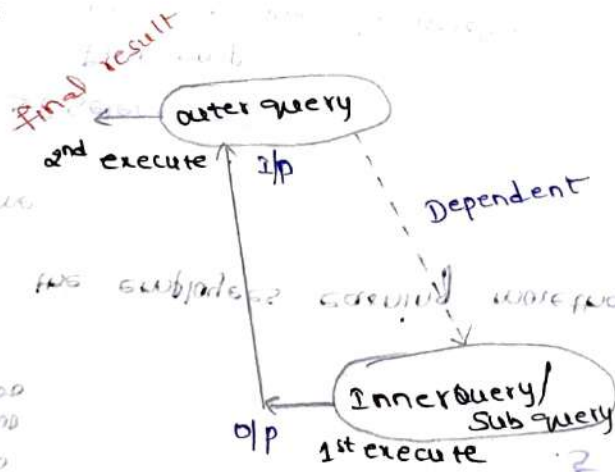
A query inside another query is known as sub query.

Working principle of sub query:-

→ Let us consider two queries Outer query and Inner query.

→ Inner query executes first and produces an output.

→ The output of inner query is given / fed as an input to outer query.



→ Then the outer query generates the result.

→ Therefore we can state that the outer query is dependent on inner query and this is the execution principle of sub query.

where and when we do use subquery.

Case i:- Whenever unknown present in the question use

Ex: Find subquery to find unknown

Ex:- waqtd name of the employee who are earning salary more than Miller.

Select ename (5000)

from emp

where sal > (select sal (3000))

800 > 3000

1500 > 3000

3000 > 3000

2500 > 3000

5000 > 3000

from emp

where ename = 'MILLER';

empno	ename	sal
1	SMITH	800
2	ALLEN	1500
3	MILLER	3000
4	SCOTT	2500
5	KING	5000

Assignment

1. waqtd name of the employees earning more than Adams.

Select ename

from emp

where sal > (select sal

from emp

where ename = 'ADAMS');

2. waqtd name and salary of the employees earning less than King.

Select ename, sal

from emp

where sal < (select sal

from emp

where ename = 'KING');

3. waqtd name and deptno of the employees if they are working in the same dept as Jones.

Select ename, deptno

from emp

where deptno = (select deptno from emp where ename = 'JONES');

4. waqtd name and job of all the employees working in the same designation as James.

Select ename, job

from emp

where job = (select job from emp where ename = 'JAMES');

5. wantd empno and ename along with annual salary of all the employees if their annual salary is greater than wards annual salary.

```
Select emp, empno, emp.ename, sal*12  
from emp  
where sal*12 > (select sal*12 from emp where ename='WARD');
```

6. wantd name and hiredate of the employees if they are hired before SCOTT

```
Select ename, hiredate  
from emp  
where hiredate < (select hiredate from emp where ename='SCOTT');
```

7. wantd name and hiredate of the employees if they are hired after the president.

```
Select ename, hiredate  
from emp  
where hiredate > (select hiredate  
from emp  
where job='PRESIDENT');
```

8. wantd name and sal of the employee if they are earning sal less than the employee whos empno is 7839

```
Select ename, sal  
from emp  
where sal < (select sal from emp  
where empno = 7839);
```

9. wantd all the details of the employees of the employees are earn: hired before miller.

```
Select *  
from emp  
where hiredate < (select hiredate from emp  
where ename = 'MILLER');
```

10. wantd ename and empno of the employees if employees are earning more than Allen.

```
Select ename and empno  
from emp  
where sal > (select sal from emp  
where ename = 'ALLEN');
```

11. Wqytd ename and salary of all the employees who are earning more than miller But less than Allen.

```
select ename, sal  
from emp where sal > (select sal from emp where ename = 'MILLER')  
and sal < (select sal from emp where ename = 'ALLEN');
```

12. Wqytd all the details of the employees working in dept 20 and working in the same designation as Smith

```
select *  
from emp where deptno = 20 and job = (select job from emp  
where ename = 'SMITH');
```

13. Wqytd all the details of the employees working as manager in the same dept as turner.

```
select *  
from emp  
where job = 'MANAGER' and  
deptno = (select deptno from emp where ename = 'TURNER');
```

14. Wqytd name and hiredate of the employees hired after 1980 and before King.

```
select ename, hiredate  
from emp
```

```
where hiredate > '31-DEC-1980' and  
hiredate < (select hiredate from emp where ename = 'KING');
```

15. Wqytd name and sal along with Annual sal for all employees whose sal is less than Blake and more than 3500.

```
select ename, sal,  
select emp.ename, emp.sal, (emp.sal * 12) from emp  
where sal < (select sal from emp where ename = 'BLAKE') and  
sal > 3500
```

16. Wqytd all the details of employees who are earn more than Scott But less than King.

```
select * from emp  
where sal > (select sal from emp where ename = 'SCOTT') and
```

```
sal < (select sal from emp where ename = 'KING');
```

17. Wqtd name of the employees whose name starts with 'A' and works in the same dept as Blake

Select ename

from emp

where ename like 'A%' and deptno = (select deptno from emp where ename = 'BLAKE');

18. Wqtd name and comm if employees earn commission and work in the same designation as Smith

select ename, comm

from emp

where comm >= 0 and job = (select job from emp where ename = 'SMITH');

19. Wqtd details of all the employees working as clerk in the same dept as Turner.

Select *

from emp

where job = 'CLERK' and deptno = (select deptno from emp

where ename = 'TURNER');

20. Wqtd ename, sal and designation of the employees whose annual salary is more than Smith and less than King.

Select ename, sal, job

from emp

where sal * 12 > (select sal * 12 from emp where ename = 'SMITH') and

sal * 12 < (select sal * 12 from emp where ename = 'KING');

21/07/24

1. Wqtd details of employee whose sal is more than Miller and less than KING

Select *

from emp

where sal > (select sal from emp where ename = 'MILLER') and

sal < (select sal from emp where ename = 'KING');

2. Write name of the employee who are earning same salary.

```

Select ename
from emp
where sal in (select sal
              from emp
              group by sal
              having count(sal) > 1);

```

3. Write details of the employee whose name start with 'A' or whose name does not start with 'O' and hired before Smith and after Blake.

```

Select *
from emp
where (ename like 'A%') and (ename not like 'O%') and hiredate <
      (select hiredate from emp where ename = 'SMITH') and
      hiredate > (select hiredate from emp where ename = 'BLAKE');

```

Case-II

→ Whenever data to be selected and the condition to be execute are present in different table we use Sub-query

Ex:- Write name of the employee whose salary more than 3000.

```

select dname Accounting
from emp
where deptno = (select deptno
                from emp
                where sal > 3000);

```

Dept

Dept no	Dname	loc
10	Accounting	Mumbai
20	Operation	Goa
30	Sales	Hyderabad

Emp

empno	ename	sal	deptno
1	SMITH	1000	10
2	ALLEN	3000	20
3	MILLER	2500	30
4	SCOTT	1500	20
5	KING	5000	10
6	BLAKE	3000	30

2. Wqytd ename of the employee working in newyork.

```
Select ename  
from emp  
where deptno = (Select deptno  
from dept  
where loc = 'Newyork');
```

3. Wqytd name of the employees whose salary more than 3000 and
waking in accounting or sales.

```
Select ename  
from emp  
where sal > 3000 and deptno = (Select deptno  
from dept  
where dname = 'ACCOUNTING' or  
dname = 'SALES');
```

4. Wqytd details of the employees salary more than miller.

```
Select *  
from dept  
where dept > (Select deptno  
from emp  
where (ename = 'MILLER') sal > (select sal from emp  
where ename = 'MILLER'));
```

5. Wqytd loc of the employees who are waking in accounting

```
Select loc  
from dept  
where dname = 'ACCOUNTING';
```

6. Wqytd loc of the Smith's

```
Select loc  
from dept  
where deptno = (Select deptno  
from emp  
where ename = 'SMITH');
```

7. Wqatd dname of the employees whose salary more than Blake.

```
select dname
from dept
where deptno = (select deptno
                 from emp
                 where sal > (select sal
                             from emp
                             where ename = 'BLAKE'));
```

8. Wqatd dname and loc of the employees who are hired before Allen.

```
select dname, loc
from dept
where deptno = (select deptno
                 from emp
                 where hiredate < (select hiredate
                                   from emp
                                   where ename = 'ALLEN'));
```

Assignment

1. Wqatd dname of the employees whose name is Smith

```
select dname
from dept
where deptno = (select deptno
                 from emp
                 where ename = 'SMITH');
```

2. Wqatd dname and loc of the employee whose ename is King.

```
select dname, loc
from dept
where deptno = (select deptno
                 from emp
                 where ename = 'KING');
```

3. write loc of the emp whose employee number is 7902

Select loc

from dept

where deptno = (select deptno

from emp

where empno = 7902);

4. write dname and loc along with deptno of the employee whose name ends with 'R'.

Select dname, loc, deptno

from dept

from emp

where

deptno = (select deptno

from emp

where ename like '%R');

5. write dname of the employee whose designation is president.

Select dname

from dept

where deptno = (select deptno

from emp

where job = 'PRESIDENT');

6. write names of the employees working in accounting department.

Select ename

from emp

where deptno = (select deptno

from dept

where dname = 'ACCOUNTING');

7. write names and salaries of the employees who are working in the location Chicago.

Select ename, sal

from emp

where deptno = (select deptno

from dept

where loc = 'CHICAGO');

8. want details of the employees working in sales.

```
select *  
from emp  
where deptno = (select deptno  
from dept  
where dname = 'SALES');
```

9. want details of the employees along with annual salary if employees are working in new york.

```
select emp.*, sal * 12  
from emp  
where deptno = (select deptno  
from dept  
where loc = 'NEW YORK');
```

10. want details of employees working in operations department.

```
select ename  
from emp  
where deptno = (select deptno  
from emp  
where dname = 'OPERATIONS');
```

11. want names of the employees earning more than scott in accounting department.

```
select ename  
from emp  
where sal > (select sal from emp where ename = 'SCOTT')  
and deptno = (select deptno from dept where  
dname = 'ACCOUNTING');
```

12. want details of the employees working as manager in the location chicago.

```
select *  
from emp  
where job = 'MANAGER' and deptno = (select deptno  
from dept  
where loc = 'CHICAGO');
```

13. want name and Sal of the employees who earning more than King in the dept accounting.

```
select ename, sal
from emp
where deptno = (select deptno
from dept where dname = 'ACCOUNTING')
and sal > (select sal
from emp where ename = 'KING');
```

14. want details of the employees working as Salesman in the department sales.

```
select *
from emp
where deptno = (select deptno
from dept
where dname = 'SALES') and job = 'SALESMAN';
```

15. want name, sal, job, hiredate of the employees working in operations department and hired before King.

```
select ename, sal, job, hiredate
from emp
where deptno = (select deptno
from dept
where dname = 'OPERATIONS') and hiredate < (select hiredate
from emp
where ename = 'KING');
```

16. want all the employees whose department names ending 'S'

```
select *
from emp
where deptno = (select deptno
from dept
where dname like '%S');
```

17. want dname of the employees whose names has character 'A' in it.

```
select dname
from dept
where deptno = (select deptno
from emp
where ename like '%A%');
```

18. write dname and loc of the employees whose salary is rupees 800.

```
select dname, loc
from dept
where deptno = (select deptno
from emp
where sal >= 800);
```

19. write dname of the employees who earn commission.

```
select dname
from dept
where deptno = (select deptno
from emp
where comm is not null);
```

20. write loc of the employees if they earn commission in dept 40.

```
select loc
from dept
where deptno = (select deptno
from emp
where deptno = 40 and comm is not null);
```

5/08/24

1. write ename, dname who are working in deptno = 10.

```
select ename, (select dname
from dept
where deptno = 10)
```

```
from emp
where deptno = 10;
```

o/p:-

clerk	Accounting
king	Accounting
Miller	Accounting

1. want details of the employees sal from more than all the deptno to emp

```

Select *
from emp
where deptno > (Select sal
                from deptp
                where deptno=10);

```

Types of sub query:-

i, single row sub query:

If a sub query returns exactly 1 record / 1 value

we call it as single row sub query.

If a sub query returns single value then we can use both

normal op as well as special op.

Ex:-

1. want name of the employee whose salary equals to ALLEN,

```

Select ename
from emp
where sal = (select sal
             from emp
             where ename = 'ALLEN');

```

Empno	Ename	Sal	Deptno
1	SMITH	800	10
2	ALLEN	3000	20
3	MILLER	1500	30
4	SCOTT	2500	20
5	KING	5000	10
6	SMITH	3000	30

ii, multi-row sub query:-

If a sub query returns more than 1 record / 1 value we

call it as multi Row sub query.

If a sub query returns more than 1 value then we cannot

use normal op we must only use special op.

Ex:-

want name of the employee whose salary to smith.

```

Select ename
from emp
where sal <= (select sal
              from emp
              where ename = 'SMITH');

```

use only
IN operator

equal will accept
only one value at RHS.

6/08/24

Subquery :-

- 1. All
- 2. Any
- 3. Exist
- 4. Not exist

Note:- IN & NOT IN can be used as a substitute for $\neq$!

Operator only not for relational op ($>$, $<$, $>=$, $<=$)

All operator:-

All operator is a sub query operator which has to be used along with relational operator which will compare the value present at the LHS with all the values of RHS.

All op returns true if all the values at the RHS have satisfied the condition.

Wanted name of the employee whose salary more than all the deptno 20.

select ename

from emp

where sal > all (select sal

from emp

where deptno = 20);

$800 > \text{all}(1500, 2000)$

$2000 > \text{all}(1500, 2000)$

$800 > 1500 (F)$

$2000 > 1500 (T)$

$800 > 2000 (F)$

$2000 > 2000 (F)$

$1500 > \text{all}(1500, 2000)$

$5000 > \text{all}(1500, 2000)$

$1500 > 1500 (T)$

$5000 > 1500 (T)$

$1500 > 2000 (F)$

$5000 > 2000 (T)$

$3000 > \text{all}(1500, 2000)$

$1000 > \text{all}(1500, 2000)$

$3000 > 1500 (T)$

$1000 > 1500 (F)$

$3000 > 2000 (T)$

$1000 > 2000 (F)$

empno	ename	sal	deptno
1	SMITH	800	10
2	ALLEN	1500	20
3	MILLER	3000	30
4	SCOTT	2000	20
5	KING	5000	10
6	ADAMS	1000	30

Any operator :-

Any operator is a subquery operator which has to be used along with relational operator which will compare the value present at the LHS with all the values of RHS.

Any op returns true if one of the values at the RHS have satisfied the condition.

1. Wanted name of the employee whose salary more than any of the

emp no	ename	sal	dept no
1	SMITH	800	10
2	ALLEN	1500	20
3	MILLER	3000	30
4	SCOTT	2000	20
5	KING	5000	10
6	ADAMS	1000	30

deptno = 20

Select ename

from emp

where sal > Any (select sal

from emp

where deptno = 20);

800 > Any (1500, 2000)

800 > 1500 (F)
800 > 2000 (F) > (F)

2000 > Any (1500, 2000)

2000 > 1500 (T)
2000 > 2000 (F) > (T)

1500 > Any (1500, 2000)

1500 > 1500 (T)
1500 > 2000 (F) > (T)

5000 > Any (1500, 2000)

5000 > 1500 (T)
5000 > 2000 (T) > (T)

3000 > Any (1500, 2000)

3000 > 1500 (T)
3000 > 2000 (T) > (T)

1000 > Any (1500, 2000)

1000 > 1500 (F)
1000 > 2000 (F) > (F)

Flow of subquery :-

What is subquery?

Show the execution / Draw it

when?

Case 1

Case 2

(1) Wanted details of the employee whose name is end with character 'e' who are hired after any of the Salesman.

Types

single row

multi row

All

Any

Select *

from emp

where ename like '%E' and hiredate > Any (select

hiredate from emp

where job = 'SALESMAN');

Assignment

1. want details of the emp whose name is end with character 'ed' who are hired after any of the Salesman.

```
select *
from emp
where hiredate > Any (select hiredate
                      from emp
                      where job = 'SALESMAN');
```

2. want details of the employees hired after all the clerks.

```
select *
from emp
where hiredate > All (select hiredate
                     from emp
                     where job = 'CLERKS');
```

3. want name and sal for all the emp if they are earning less than atleast a manager.

```
select ename, sal
from emp
where sal < Any (select sal
                 from emp
                 where job = 'MANAGER');
```

4. want name and hiredate of employees hired before all the managers.

```
select ename, hiredate
from emp
where hiredate < All (select hiredate
                     from emp
                     where job = 'MANAGER');
```

5. wanted names of the employees hired after all the managers and earning salary more than all the clerks

Select ename.

from emp

where hiredate > All(select hiredate

from emp

where job='MANAGER') and sal > (select sal

from emp

where job='CLERKS');

6. wanted details of the emp working as clerk and hired before

at least a Salesman.

select *

from emp

where job='CLERK' and hiredate < Any(select hiredate

from emp

where job='SALESMAN');

7. wanted details of the emp working in accounting or Sales dept.

select *

from emp

where deptno in Any(select deptno from Dept

where dname='Accounting' or

dname='Sales');

8. wanted department names of the employees with name Smith, King,

and Miller.

Select dname

from dept

where deptno in Any(select deptno

from emp

where ename='SMITH' or ename='KING'

or ename='MILLER');

9. want details of the employees with newyork or chicago.

Select *

from emp

where deptno in (select deptno

from emp

where loc = 'NEWYORK' OR loc = 'CHICAGO');

10. want emp ename if emp are hired after all the emp

of dept 10

select ename

from emp

where hiredate > All (select hiredate

from emp

where deptno = 10);

from emp

select *

10/24

1. want name of the emp whose sal is max.

Select ename

from emp

where sal = (select max(sal)

from emp);

from emp

2. want name of the emp whose salary more than avg sal

select ename

from emp

where empno in (select empno from emp,

sal

where sal > avg(sal));

3. want ename of the emp who are hired before all (first emp).

Select ename

from emp

where empno in (select min(empno) from emp);

4. waqtd name and hiredate of the employees hired at the last.

```
Select ename, hiredate
```

```
from emp
```

```
where empno in (select max(empno)  
from emp);
```

5. waqtd name, comm of the employee who earns min comm

```
Select ename, comm
```

```
from emp
```

```
where comm in (select min(comm)  
from emp);
```

6. waqtd name, sal and comm of the employee earning max commission

```
Select ename, sal, comm
```

```
from emp
```

```
where comm in (select max(comm)  
from emp);
```

7. waqtd details of the employee who has greatest empno

```
Select *
```

```
from emp
```

```
where empno in (select max(empno)  
from emp);
```

8. waqtd details of the employee having least hiredate.

```
Select *
```

```
from emp
```

```
where hiredate = (select min(hiredate)  
from emp);
```

9. waqtd details of the employee who earning least annual sal.

```
Select *
```

```
from emp
```

```
where sal in (select min(sal)  
from emp);
```

10. Wagt'd name, Annual salary of the employees if their annual salary is more than all the salesman.

```

Select ename, sal*12
from emp
where sal*12 > (select max(sal*12)
                from emp
                where job = 'SALESMAN');

```

8/08/24

Nested subquery

→ 'A subquery written inside another subquery' is known as Nested subquery.

→ we can nest about 255 sub queries.

① wage & 3rd max sal of the employee.

select max(s₁) ^{200 1st salary}
from emp ^{200 2nd salary}
where s₁ < (select max(s₁))

$800 < 3000$
 $13000 < 3000$
 $3000 < 3000$
 $2000 < 3000$
 $5000 < 3000$
 $13000 < 3000$

from emp

where $s.d < (select \max(s.d)$
800 < 5000 from emp); 4th min
1300 < 5000
3000 < 5000
2000 < 5000
5000 < 5000
1300 < 5000

empno	ENAME	Sal	
1	SMITH	800	1st min 5th max
2	ALLEN	1300	4th max 2nd min
3	MILLER	3000	2nd max
4	SCOTT	2000	3rd min 3rd max
5	KING	5000	1st max
6	ADAMS	1300	4th max 2nd min

2. ^{20/10/2021} way to find mines -

Select min(ssd) subpdses and one covered by min

3. Wanted to find 3rd min of the employee

select min(sal)

from emp
where sal > (select min(sal)

$800 > 1300$
 $1300 > 1300$
 $3100 > 1300$
 $2000 > 1300$
 $5000 > 1300$
 $1300 > 1300$

from emp
where sal > (select min(sal)
from emp));

$$\begin{array}{l} \boxed{800 > 800} \\ 1300 > 800 \\ 3000 > 800 \\ 2000 > 800 \\ 5000 > 800 \\ 1300 > 800 \end{array}$$

4. Wqtd name of the employees who are earning 3rd max salary.

```
select ename  
from emp  
where sal in (select max(sal)  
              from emp  
              where sal < (select max(sal)  
                           from emp  
                           where sal < (select max(sal)  
                                         from emp))));
```

5. Wqtd dname of the employees who are earning 4th min salary.

```
select dname from dept  
where deptno in (select deptno from emp  
                 where sal < (select min(sal) from emp  
                             where sal > (select max(sal) from emp  
                                           where sal > (select max(sal)  
                                                         from emp where sal > (select  
                                                             max(sal) from emp))));
```

6. Wqtd details of the employees who are joined 5th

```
select * from emp where sal > (select min(sal) from emp  
                                where hiredate > (select min(hiredate) from emp  
                                                    where hiredate > (select min(hiredate) from emp  
                                                                    where hiredate > (select min(hiredate) from emp  
                                                                        where hiredate > (select min(hiredate) from emp))));
```

7. Wqtd dept details of the employees who are hired 4th.

```
select * from dept  
where deptno in (select deptno from emp  
                 where hiredate > (select min(hiredate) from emp  
                                     where hiredate > (select min(hiredate) from emp  
                                                         where hiredate > (select min(hiredate) from emp  
                                                             where hiredate > (select min(hiredate) from emp))));
```

8. waqtd name of the employees 7th min salary.

```
Select ename from emp
```

```
where sal > (select min(sal) from emp)
```

```
where sal > (select min(sal) from emp
```

```
where sal > (select min(sal) from emp
```

```
where sal > (select min(sal) from emp
```

```
where sal > (select min(sal) from emp
```

```
where sal > (select min(sal) from emp
```

```
where sal > (select min(sal) from emp))))))));
```

Assignment

1. waqtd 2nd minimum salary.

```
Select min(sal)
```

```
from emp
```

```
where sal > (select min(sal)
```

```
from emp));
```

2. waqtd 5th maximum salary.

```
Select max(sal)
```

```
from emp
```

```
where sal > (select max(sal)
```

```
from emp
```

```
where sal > (select max(sal)
```

```
from emp
```

```
where sal > (select max(sal) from emp
```

```
where sal > (select max(sal) from emp)))));
```

3. waqtd name of the employee who are earning 3rd max sal.

```
Select ename (select max(sal) from emp)
```

```
from emp
```

```
where sal > (select max(sal) from emp
```

```
where sal > (select max(sal) from emp
```

```
where sal > (select max(sal) from emp)))));
```

4. waqtd empno of the employee earning 2nd maximum salary.

```
Select empno
```

```
from emp
```

```
where sal in (select max(sal) from emp
```

```
where sal < (select max(sal) from emp));
```

Sad.

Where deptno in (select max(sal) from emp
deptno)

Where $sal \leftarrow (select \max(sal) \text{ from emp})$

where $saf < (\text{select max}(saf) \text{ from emp})$

6. Wage details of the employee who has hired 2nd

select * from emp
where hiredate > (select min(hiredate) from emp)

where hiredate > (select min(hiredate) from emp));

last employee

Where hiredate < (select max(hiredate))

and loc of the employee who was hired first.

select * from emp
where hiredate in (select min(hiredate)
from emp)

select * from emp

select * from emp
where sal > (select min(sal) from emp)

where $saf > (select \min(saf) \text{ from emp})$

where $sal > C(select \min(sal) \text{ from emp})$

where $s \geq \min(s)$ from emp

where sal > (select min(sal) from emp)

Where sal > (select min (sal) from emp)

```
where sals (select min(sal) from emp))))))));
```

10. Write a query of the employee getting maximum salary

```

Select dname
from dept
Where deptno in (select deptno from emp
Where sal >= (select max(sal) from emp
Where sal < (select max(sal) from emp))) ;

```

9/08/24

Employees & Manager Relation

emp no	ename	mgr
1	SMITH	5
2	ALLEN	1
3	MILLER	2
4	SCOTT	5
5	KING	NULL
6	ADAMS	1

empno: 01
ename: DINGI
mgr: 01

empno: 01
ename: CEO
mgr: null

empno: 03
ename: PINGA
mgr: 02

empno: 06
ename: GINGI
mgr: 02

empno: 05
ename: PINGA
mgr: 02

empno: 10
ename: DINGA
mgr: 03

empno: 04
ename: PINGI
mgr: 03

empno: 07
ename: BINGA
mgr: 04

empno: 08
ename: LINGA
mgr: 01

empno: 07
ename: LINGI
mgr: 05

To find the manager/repating manager

Empno = mgr

Select ename

DINGA

```

from emp
where empno in (select mgr
from emp
where ename = 'DINGA');

```

Select ename

SMITH

```

from emp
where empno in (select mgr
from emp
where ename = 'ADAMS');

```

To find the employees/repaters

mgr = empno

Select ename

LINGI

```

from emp
where mgr in (select empno
from emp
where ename = 'PINGA');

```

Select ename

SMITH
SCOTT

```

from emp
where mgr in (select empno
from emp
where ename = 'KING');

```

1. Wqatd name of the Allen's manager (details)

Select ~~ename~~,*

from emp

where mgr empno in (select mgr

from emp

where ename='ALLEN');

2. Wqatd name of the employees who are reporting to Blake

Select ename

from emp

where mgr in (select empno

from emp

where ename='BLAKE');

3. Wqatd details of the Miller manager's manager

Select *

from emp

where

empno in (select mgr

from emp

where empno in (select mgr

from emp

where ename='MILLER');

4. Wqatd details of the emp Smith's manager's manager

Select *

from emp

where empno in (select mgr

from emp

where empno in (select mgr

from emp

where ename='SMITH');

5. Wqatd ename and salary of the employee who are reporting to Miller's manager

from emp

Select ename, sal

from emp

where mgr in (select empno

from emp

where mgr empno in (select mgr

from emp

where ename = 'MILLER'));

6. Wqtd name of the employees Adams manager's manager -

select dname

from dept

where deptno in (select deptno

from emp

where

where empno

in (select mgr

from emp

where empno

in (select mgr

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

from emp

where ename = 'ADAMS'));

7. Wqtd. name of the Scott manager's manager name

Select ename

from emp

where empno in (select mgr

from emp

where empno in (select mgr

from emp

where ename = 'SCOTT'));

from emp

where ename = 'SCOTT'));

from emp

where ename = 'SCOTT'));

from emp

where ename = 'SCOTT'));

from emp

where ename = 'SCOTT'));

from emp

where ename = 'SCOTT'));

from emp

where ename = 'SCOTT'));

Assignment:-

1. Wqtd Smiths Reporting manager's name.

Select ename in (select mgr

from emp

where empno in (select mgr

from emp

where ename = 'SMITH'));

2. Wqytd Adams manager's manager name.

```
select ename  
from emp  
where empno in (select mgr  
from emp  
where empno in (select mgr  
from emp  
where ename = 'ADAMS'));
```

3. Wqytd dname of the Jones manager.

```
select dname  
from dept  
where deptno in (select deptno  
from emp  
where empno in (select mgr  
from emp  
where ename = 'JONES'));
```

4. Wqytd Miller's manager's salary.

```
select sal  
from emp  
where empno in (select mgr  
from emp  
where ename = 'MILLER');
```

5. Wqytd loc of Smith's manager's manager.

```
select loc  
from emp  
where empno in (select mgr  
from emp  
where empno in (select mgr  
from emp  
where ename = 'SMITH'));
```

6. Wqytd name of the employees reporting to Blake.

```
select ename  
from emp  
where mgr in (select empno  
from emp  
where ename = 'BLAKE');
```

7. Wqytd number of employees reporting to King.

```
select count(*)  
from emp  
where mgr in (select empno  
from emp  
where ename = 'KING');
```

8. want details of the employees reporting to jones.

```
select *  
from emp  
where mgr in (select empno  
from emp  
where ename='JONES');
```

9. want enames of the employees reporting to blake's manager.

```
select ename  
from emp  
where empno in (select empno  
from emp  
where mgr in (select mgr  
from emp  
where ename='BLAKE'))
```

10. want number of employees reporting to ford's manager

```
select count(*)  
from emp
```

11. want mgr in (select empno

```
from emp  
where empno in (select mgr  
from emp  
where ename='FORD'))
```

12/08/24

JOINS - The process of retrieving the data from multiple tables

Simultaneously is known as joins.

Types of joins

we have 5 types of joins

1. Cartesian join / Cross join

2. Inner join / Equi join

3. Outer join

4. i. Left outer

ii. Right outer join

iii. Full outer join

4. Self join

5. Natural join.

1. Cartesian Join

In Cartesian join a record from table 1 will be merged with all the records of table 2.

Syntax:-

1. ANSI [American national Standard Institute]

Select column_name

from table_name1 cross join table_name2;

2. Oracle

Select column_name

from table_name1, table_name2;

1. Write name and dept name of all the employee.

ANSI:-

Select ename, dname

from emp cross join dept;

Oracle:-

Select ename, dname

from emp, dept;

Emp	Dept
Ename	Deptno
A	10
B	20
C	10

Result:-

ename	emp. deptno	dept. deptno	Dname
A	10	10	D1
A	20	20	D2
A	10	30	D3
B	20	10	D1
B	20	20	D2
B	20	30	D3
C	10	10	D1
C	10	20	D2
C	10	30	D3

Number of col = Number of col T₁ + Number of col T₂

2 + 2 = 4 columns

2 + 2 = 4 columns

Number of rows in the result table will be equivalent to the product of number of row present in the both the tables.

Number of rows = number of rows T₁ × Number of rows T₂

3 × 3

= 9 rows.

If there are 3 rows in table 1 and 3 rows in table 2, then the result will have 9 rows.

Inner join:-

It is used to obtain only matching records. (Or) A records which has a pair.

<Join Condition> Syntax:-

Table-name1.col-name = Table-name2.col-name

Syntax for inner join:-

1. ANSI [American National Standard Institute]

Select col-name

from Table-name1 inner join Table-name2

on <join-condition>;

2. Oracle

Select col-name

from Table-name1, Table-name2

where <join-condition>;

→ write down the name and dept name of all the employees.

ANSI

Select ename, dname

from emp inner join dept

on emp.deptno = dept.deptno;

Oracle

Select ename, dname

from emp, dept

where emp.deptno = dept.deptno;

EMP

ename	deptno
A	10
B	20
C	30
D	10

dept

deptno	dname
10	D1
20	D2
30	D3
40	D4

Result:-

ename	emp.deptno	Dept.deptno	dname
A	10	10	D1
B	20	20	D2
C	30	30	D3
D	10	10	D1

→ write down the name and loc of the employees who are working as manager.

ANSI Select ename, loc

from emp inner join dept

on emp.deptno = dept.deptno

where job = 'MANAGER';

Oracle

Select ename, loc

from emp, dept

where job = 'MANAGER' and
dept.deptno = emp.deptno;

2. wanted emp. name, and dept details of the employees who are working in newyork

ANSI Select ename, dept, *
from emp inner join dept
on emp.deptno = dept.deptno
where loc = 'NEW YORK';

Oracle
Select ename, dept, *
from emp, dept
where loc = 'NEW YORK' and
emp.deptno = dept.deptno;

3. wanted ename, dname, loc of the employees whose salary more than miller

ANSI Select ename, dname, loc
from emp inner join dept
on emp.deptno = dept.deptno
where sal > (select sal
from emp
where ename = 'MILLER');

Oracle: Select ename, dname, loc
from emp, dept
where emp.deptno = dept.deptno
and sal > (select sal
from emp
where ename = 'MILLER');

4. wanted emp details and dname of the employees who are working as manager or salesman and in 10 or 30

ANSI
Select emp.*, dept.*
from emp inner join dept
on dept.deptno = emp.deptno
where job in ('SALESMAN', 'MANAGER')
and emp.deptno in (10, 30)

Oracle:
Select emp.*, dept.*
from emp, dept
where emp.deptno = dept.deptno
and job in ('MANAGER', 'SALESMAN')
and emp.deptno in (10, 30)

5. wanted ename, deptno and dname of the employees whose salary less than scott

ANSI
Select ename, deptno, dname
from emp inner join dept
on emp.deptno = dept.deptno
where sal < (select sal
from emp
where ename = 'SCOTT');

Oracle:
Select ename, emp.deptno, dname
from emp, dept
where dept.deptno = emp.deptno
and sal < (select sal
from emp
where ename = 'SCOTT');

Natural join:-

'It behaves a Inner join if there is a relation between the given two tables, otherwise it behaves as cross join'.

Syntax:- ANSI:-

Select col-name

from table-name1 Natural join Table-name2;

Ex:-

Emp		Dept	
ename	deptno	deptno	dname
A	10	10	D1
B	20	20	D2
C	30	30	D3
D	10	40	D4

Note:- 'If there is a relation b/w two tables, result will be

Result:-

ename	deptno	dname
A	10	D1
B	20	D2
C	30	D3
D	10	D1

Customer It acts like inner join

CID	ename
1001	X
1002	Y

Note:- If there is no relation b/w two tables, result will be

Emp

ename	deptno
A	10
B	20
C	30
D	40

acts like Cartesian join

Result:-

ename	dname	CID	ename
A	10	1001	X
A	10	1002	Y
B	20	1001	X
B	20	1002	Y
C	30	1001	X
C	30	1002	Y
D	10	1001	X
D	10	1002	Y

Assignment

1. Name of the employee and his location of all the employees.

Select ename, loc

from emp, dept

where emp.deptno = dept.deptno;

Select ename, loc

from dept inner join emp

on emp.deptno = dept.deptno;

2. Writed dname and salary for all the employee working in Accounting.

```
select dname, sal
```

```
from emp, dept
```

```
where emp.deptno = dept.deptno and dname = 'ACCOUNTING';
```

3. Writed dname and Annual salary for all the employees whose salary is more than 2340

```
select dname, sal*12
```

```
from dept, emp
```

```
where emp.deptno = dept.deptno and sal > 2340;
```

4. Writed ename and dname for all employees having character 'A' in their dname.

```
select ename, dname
```

```
from emp, dept
```

```
where emp.deptno = dept.deptno and dname like '%A%';
```

5. Writed ename and dname for all the employees working as Salesman.

```
select ename, dname
```

```
from emp, dept
```

```
where emp.deptno = dept.deptno and job = 'SALESMAN';
```

6. Writed dname and job for all the employees who's job and dname starts with character 'S'?

```
select dname, job
```

```
from emp, dept
```

```
where emp.deptno = dept.deptno and job like 'S%' and dname like 'S%';
```

7. Writed dname and mgr no for employees who are Reporting to 7839

```
select dname, mgr
```

```
from emp, dept
```

```
where emp.deptno = dept.deptno and
```

```
mgr = 7839;
```

8. want dname and hiredate for employees hired after 83 into Accounting or Research dept.

```
select dname, hiredate
from emp, dept
where emp.deptno = dept.deptno and hiredate > '83' and
dname in ('ACCOUNTING', 'RESEARCH');
```

9. want ename and dname of the employees who are getting Comm in dept 10 or 30

```
select ename, dname
from emp, dept
where emp.deptno = dept.deptno and comm is not null
and emp.deptno in (10, 30);
```

10. want dname and empno for all employees who are empno are (7839, 7902) and are working in loc New York.

```
select dname, empno
from dept, emp
where emp.deptno = dept.deptno and empno in (7839, 7902) and
loc = 'NEW YORK';
```

15/08/24

3. Outer join :- It is used to obtain un-matched records.

1. Left outer join :-

It is used to obtain un-matched records of left table along with matching records.

Syntax :-

Left outer join condition :- Emp. Deptno = Dept. Deptno (+)

Oracle :-

select col-name

from Table-name1, Table-name2

where <join condition>;

ANSI :-

select col-name

from Table-name1 left outer join Table-name2

on <join condition>;

Ex:- Select *

from emp, dept

where emp.deptno = dept.deptno(+);

Result:-

emp

ename	deptno
A	10
B	NULL
C	20
D	NULL

dept

deptno	dname
10	D1
20	D2
30	D3

ename	emp. deptno	dept. deptno	dname
A	10	10	D1
C	20	20	D2
B	NULL	NULL	NULL
D	NULL	NULL	NULL

2. Right outer join:- It is used to obtain un-matched records of right table along with matched record.

Condition:- emp.deptno(+) = dept.deptno

Syntax:- oracle:- Select col-name
from emp, dept (Table-name1, Table-name2)
where <join condition>

ANSI:- Select col-name
from emp Right outer join dept
on <join condition>

Query

Ex:- Select *

from emp, dept

where emp.deptno(+) = dept.deptno

from emp right outer join dept
on emp.deptno = dept.deptno

Result:-

emp

ename	deptno
A	10
B	NULL
C	20
D	NULL

dept

deptno	dname
10	D1
20	D2
30	D3

ename	emp. deptno	dept. deptno	dname
A	10	10	D1
C	20	20	D2
NULL	NULL	30	D3

3. Full outer join:- It is used to obtain un-matched records of both left table and right table with matched record.

Condition:-

Syntax:- ANSI:- Select col-name
from table-name1 Full outer join Table-name2
on <join condition>

1. waqtd

Ex:- Query

Select *

from emp full outer join dept

on emp.deptno = dept.deptno

emp	dept
ename	deptno
A	10
B	NULL
C	20
D	NULL

Result:-

emp. ename	dept. deptno	dept. deptno	dname
A	10	10	D1
C	20	20	D2
B	NULL	NULL	NULL
D	NULL	NULL	NULL
NULL	NULL	30	D3

4. SELF JOIN:-

Joining itself is known as Self join (or)

Joining will happen with the same table, is known as self join.

Syntax:-

Oracle:- select column_name

from Table_name1 [ALIAS], Table_name2 [ALIAS]

where <join condition>

condition:- $E_1.mgr = E_2.empno$

ANS:- select column_name

from Table_name1 [ALIAS] join Table_name2 [ALIAS]

on <JOIN condition>

Note:- Whenever we want different record we in the same

table we use self join.

1. waqtd emp name and manager's name of all the employees.

select e1.ename, e2.ename

from emp E1, emp E2

where $E_1.mgr = E_2.empno$

empno	ename	mgr
1	SMITH	5
2	ALLEN	1
3	SCOTT	4
4	MILLER	2
5	KING	NULL

empno	ename	mgr
1	SMITH	5
2	ALLEN	1
3	SCOTT	4
4	MILLER	2
5	KING	NULL

Result:-

E1. empno	E1. ename	E1. mgr	E2. empno	E2. ename	E2. mgr
1	SMITH	5	5	KING	NULL
2	ALLEN	1	1	SMITH	5
3	SCOTT	4	4	MILLER	2
4	MILLER	2	2	ALLEN	1

Assignment:-

1. Wqytd name of the employee and his manager's name if employee is working as clerk

```
select e1.ename, e2.ename
from emp e1, emp e2
where e1.mgr = e2.empno and
      e1.job = 'CLERK';
```

2. Wqytd name of the employee and manager's designation if manager works in dept. 10 or 20

```
select E1.ename, E2.job
from emp E1, emp E2
where E1.mgr = E2.empno
and E2.deptno in (10, 20);
```

3. Wqytd name of the employee and manager salary if employee and manager both earn more than 2300.

```
select e1.ename, e2.sal
from emp e1, emp e2
where e1.mgr = e2.empno and e1.sal > 2300 and e2.sal > 2300;
```

4. Wqytd emp name and manager's hiredate if employee was hired before 1982

```
select e1.ename, e2.hiredate
from emp e1, emp e2
where e1.mgr = e2.empno and e1.hiredate < '1982';
```

5. Wqytd emp name and manager's Comm if employee works as salesman and manager works in dept 30.

```
select e1.ename, e2.comm
from emp e1, emp e2
where e1.mgr = e2.empno and e1.job = 'SALESMAN' and
      e2.deptno in (30);
```

6. wantd emp name and manager name and their salaries if employee earns more than manager.

```
select e1.ename, e2.ename  
from emp e1, emp e2  
where e1.mgr = e2.empno and select e1.sal  
e1.sal > select e2.sal; from emp e2
```

7. wantd emp name and hiredate, manager name and hiredate if manager was hired before employee.

```
select e1.ename, e2.hiredate, e2.ename, e2.hiredate  
from emp e1, emp e2  
where e1.mgr = e2.empno and e2.hiredate < e1.hiredate;
```

8. wantd emp name and manager name both are working in same job.

```
select e1.ename, e2.ename  
from emp e1, emp e2  
where e1.mgr = e2.empno and e1.job = e2.job;
```

9. wantd emp name and manager name if manager is working a actual manager

```
select e1.ename, e2.ename  
from emp e1, emp e2  
where e1.mgr = e2.empno;
```

10. wantd emp name and manager name along with their annual salary if employee works in dept 10, 20 A manager salary is greater than employee salary.

```
select e1.ename, e1.sal*12, e2.ename, e2.sal*12  
from emp e1, emp e2  
where e1.mgr = e2.empno and e1.deptno in (10, 20) and  
e2.sal > e1.sal;
```

11. write employees name and manager's designation for all the employees.

```
select e.ename, e2.job
```

```
from emp e1, emp e2
```

```
where e1.mgr = e2.empno;
```

12. write employees name and manager's salary for all the employees if manager's salary ends with 50

```
select e1.ename, e2.sal
```

```
from emp e1, emp e2
```

```
where e1.mgr = e2.empno
```

```
and e2.sal like '%50';
```

13. write emp name and manager name of the employees if their employee salary more than Allen.

```
select e1.ename, e2.ename
```

```
from emp e1, emp e2
```

```
where e1.mgr = e2.empno and e1.sal > (select sal from emp where ename = 'ALLEN');
```

14. write ename and manager name of the emp if their emp sal less than 3000 and manager salary less than Blake.

```
select e1.ename, e2.ename
```

```
from emp e1, emp e2
```

```
where e1.mgr = e2.empno and e1.sal < 3000 and e2.sal < (select sal from emp where ename = 'BLAKE');
```

15. write details of the employees and manager if their emp sal more than manager.

```
select e1.*, e2.*
```

```
from emp e1, emp e2
```

```
where e1.mgr = e2.empno and e1.sal > e2.sal;
```

16. waqtd ename and hiredate, manager name and hiredate if there employee hired before the manager.

```
select e1.ename, e1.hiredate, e2.ename, e2.hiredate
from emp e1, emp e2
where e1.mgr = e2.empno and e1.hiredate < e2.hiredate;
```

17. waqtd emp name and manager name of the employee along with hike 10% annual salary.

```
select e1.ename, e2.ename, e1.sal*12 + e2.sal*12 *  $\frac{10}{100}$ 
from emp e1, emp e2
where e1.mgr = e2.empno;
```

17/08/24

1. waqtd employee name and manager name of all the employees.

```
select e1.ename, e2.ename as mname
from emp e1, emp e2
where e1.mgr = e2.empno;
```

2. waqtd ename and his salary, manager name and his salary of all the employees.

```
select e1.ename, e1.sal, e2.ename as mname, e2.sal
from emp e1, emp e2
where e1.mgr = e2.empno;
```

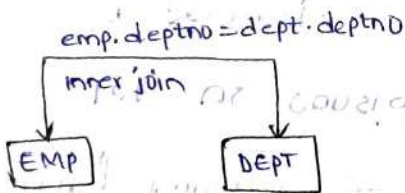
3. waqtd ename and mname along with hike the 36% their annual salary.

```
select e1.ename, e1.sal*12 + e1.sal*12*36/100 as esal,
e2.ename as mname, e2.sal*12 + e2.sal*12*36/100 as msal
from emp e1, emp e2
where e1.mgr = e2.empno;
```

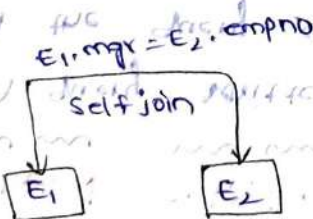
3. $mgr.dname$ and his $dname$, and manager name and his $deptname$ manager's $managername$ and manager's $managerdname$ if emp sal more than manager's $salary$, and manager's manager hired before emp and manager's $dname$ start with character 'A' or 'D' and manager's manager's manager working in same loc and manager's manager earning sal more than scott manager and emp and manager working in same dept and emp hired month of march and manager's $managername$ start with 'P', 'A', 'K' or 'I' and emp name has more than 5 letters, and manager working as Salesman and manager's manager working as actual manager and emp are not earning a $comm$ and manager's are earning $comm$ and manager's manager does not have reporting manager. emp and manager's manager reporting to same manager.

Multiple table joins :-

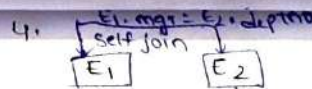
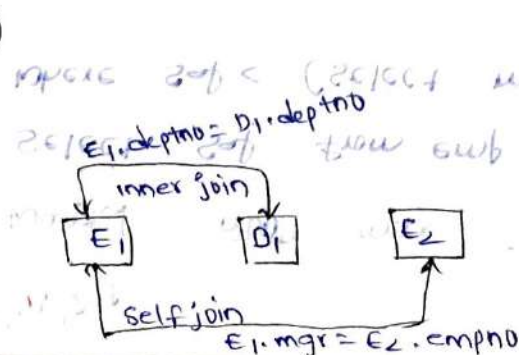
①



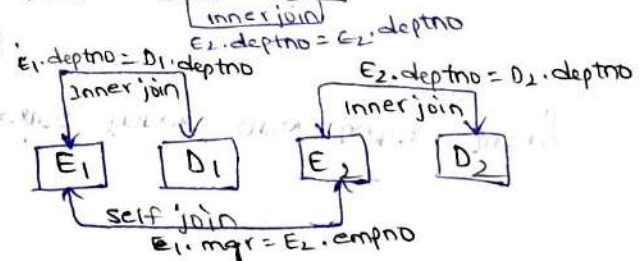
②



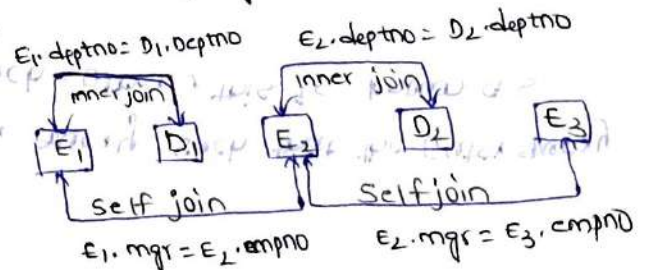
③



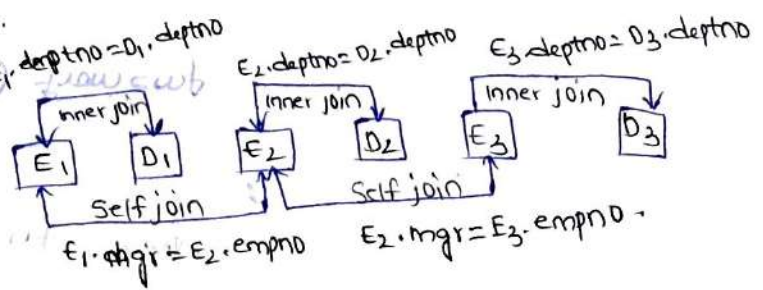
④



⑤



⑦



19/08/24

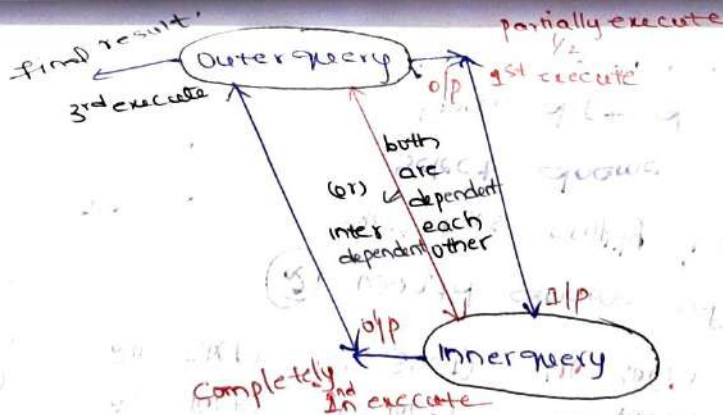
1. Wqtd 200th max salary of emp
 select sal from emp
 where sal < (select max(sal) from emp)

Co-Related Subquery:-

A query written inside another query such that the outer query and the query are dependent on each other, this is known as Co-related Subquery.

Working principle :-

Let us consider two queries inner and outer query respectively.



1. Outer query executes first but partially
2. The partially executed output is given as an input to the inner query.
3. The inner query executes completely and generates an output
4. The output of inner query is fed as an input to the outer query and outer query produces the result.
5. Therefore we can state that the outer query and the inner query both are interdependent (depend on each other).

Ex:-

Emp	dept
ename deptno	deptno dname
A 10	10 d1
B 20	20 d2
C 10	30 d3
D 30	40 d4

1. Wagt dname of the employees whose dept has working employees.

Select dname
from dept d
where deptno in (select e.deptno
from Emp E
where d.deptno = E.deptno);

10
20
30
NULL

Step No:- 1

10 = 10 (T)
10 = 20 (F)
10 = 10 (T)
10 = 30 (F)

20 = 10 (F)
20 = 20 (T)
20 = 10 (F)
20 = 30 (F)

30 = 10 (F)
30 = 20 (F)
30 = 10 (F)
30 = 30 (T)

40 = 10 (F)
40 = 20 (F)
40 = 10 (F)
40 = 30 (F)

20 = 20
30 = 30
40 = NULL

10
20
30
40

Step No:- 2

(10, 10)
20, 2
30
NULL

o/p:- d1
d2
d3

2) Wagt dname of the employee whose dept has empty i.e not working any one

Select dname
from dept d

where deptno not in (select E.deptno
from Emp E
where d.deptno = E.deptno);

o/p:- d4

20/08/24:-

EXISTS & NOT EXISTS :-

EXISTS operator:- 'Exists op is a unary op (one operand) which can accept one operand towards RHS and that Operand has to be a co-related sub query'

Exists op returns True if the Sub query returns any value other than NULL,

Ex:- Select dname

from dept d
where exists (select e.deptno
from Emp E
where d.deptno = e.deptno);

10
20
30
NULL

o CO-related sub query
occurs as obvious
not exists ob d2 20, where d.deptno = e.deptno;
d3 30

Not exists:-

Not exists op is a unary op (one operand) which can accept one operand towards RHS and that operand has to be a co-related sub query.

Not exists op returns true if the subquery returns

NULL.

Ex: select dname

from dept d

where not exists (select e.deptno

from emp e

where d.deptno = e.deptno);

To find maximum salary:-

Syntax:-

SELECT sal
from emp e₁
where (select count (distinct e₂.sal)

from emp e₂

where e₁.sal < e₂.sal) < N-1;

emp e₁
800
1200
3000
2000
5000
3500

emp e₂
800
1200
3000
2000
5000
3500

e ₂ .sal	800	1200	3000	2000	5000	3500
e ₁ .sal	F	T	T	T	T	T
800	F	F	F	F	F	F
1200	F	F	F	F	F	F
3000	F	F	F	F	F	F
2000	F	F	F	F	F	F
5000	F	F	F	F	F	F
3500	F	F	F	F	F	F

select sal
from emp e₁
where (select count (distinct sal)

from emp e₂
where e₁.sal < e₂.sal)
(2-1);
3

o/p:- 2000

To find n^{th} minimum salary:-

Syntax:-

Select sal

from emp e₁

where (Select count (distinct sal)
from emp e₂
where (e₁.sal > e₂.sal) = N-1;

Emp e₁

Emp e₂

E ₁ .sal
800
1200
3000
2000
5000
3500

E ₂ .sal
800
1200
3000
2000
5000
3500

E ₂ .sal	800	1200	3000	2000	5000	3500
E ₁ .sal						
800	F	F	F	F	F	F
1200	T	F	F	F	F	F
3000	T	T	F	T	F	F
2000	T	T	F	F	F	F
5000	T	T	T	T	T	T
3500	T	T	T	T	F	F

To find 4th min sal

Select sal

from emp e₁

where (Select count
(distinct e₂.sal

from emp e₂

where e₁.sal > e₂.sal
(4-1)
3

o/p:- 3000

21/08/24

Single row function:-

1. length() 15, TO_CHAR()

2. concat() 16, NVL() & direct stmt

3. upper() 17, length():-

4. lower() 18, It is used to count the number of

5. InitCap() 19, characters present in the given string.

6. Reverse()

7. Substr() 20, Syntax:- length('string')

8. Instr()

9. Replace() 21, Select length('Naga')

10. Mod() 22, from dual;

11. Trunc(), Concat():-

12. Round() 23, It is used to join the given strings

13. Months-between() 24, Syntax:- concat('string1', 'string2')

14. Last-day()

3. Upper() :-

It is used to convert a given string to upper case"

Syntax :- Upper('string')

4. lower() :-

It is used to convert a given string to lower case"

Syntax :- lower('string')

5. initcap() :-

It is used to convert a given string to initial capital letter case.

Syntax :- initcap('string')

6. Reverse() :-

It is used to reverse a given string.

Syntax :- Reverse('string')

7. Substr() :-

It is used to extract a part of string from the given original string.

Syntax :- Substr('original-string', position[length])

Ex :-

Substr('Butterfly', 1) → Butter fly

Substr('Butterfly', 4, 3) → But

Substr('Butterfly', -4, 3) → r fly

Substr('Butterfly', 7, 10) → fly

Substr('Butterfly', 9, 3) → tte

1 2 3 4 5 6 7 8 9
Butter fly

1. Write a query to find the name of the emp whose name starts with char 'A'.

Select ename

from emp

where substr(ename, 1, 1) = 'A';

2. Wqtd details of the employees whose name ends with vowels.

```
Select *  
from emp  
where substr(ename, -1) in ('A', 'E', 'I', 'O', 'U');
```

Wqtd the first half of the emp.

```
Select ename, substr(ename, 1, (length(ename)/2)) from emp;
```

Wqtd the second half of the emp.

```
Select ename, substr(ename, (length(ename)/2+1)) from emp;
```

22/08/24

8. INSTR() :-

It is used to obtain the position in which the string is present in the original string.

Syntax :-

EX:- $\text{Instr}(\text{'original-string'}, \text{'string'}, \text{position}, [\text{occurrence}])$

$\text{Instr}(\text{'MALAYALAM'}, \text{'A'}, 1) \Rightarrow 2$

$\text{Instr}(\text{'MALAYALAM'}, \text{'A'}, 1, 2) \Rightarrow 4$

$\text{Instr}(\text{'MALAYALAM'}, \text{'L'}, 2, 2) \Rightarrow 7$

$\text{Instr}(\text{'MALAYALAM'}, \text{'M'}, 2, 3) \Rightarrow 0$

$\text{Instr}(\text{'MALAYALAM'}, \text{'A'}, 1, 5) \Rightarrow 0$

9. REPLACE() :-

It is used to replace a string with another string in the original string.

Syntax :-

$\text{Replace}(\text{'original-string'}, \text{'string'}, [\text{'new-string'}])$

EX:- $\text{Wqtd}(\text{'2's'}) \Rightarrow 1$

Select Replace('Butterfly', 'B', 'C')

From dual;

$\rightarrow \text{select replace('Butterfly', 'B')}$

from dual;

o/p:- Cutterfly.

o/p:- utterfly.

10. Mod() :-

It is used to obtain modulus/remainder of the given number.

Syntax :-

mod(M, N)

Ex :- mod(5, 2) → 1

Select mod(5, 2)

from dual;

11. Round() :-

It is used to round-off the given number based on the scale

Value

Syntax :-

Round(Number, [scale])

Ex :-

Round(5, 2) # 5

Round(5.7) # 6

Round(5.5) # 6

Round(27.9) # 28

Round(9494.2) # 9494

Round(9494.5) # 9495

Round(00025)

Round(00024.5) # 25

→ Round(2454.5789, 1) ⇒ 2454.6

Round(2454.5289, 1) ⇒ 2454.5

Round(2454.8289, 2) ⇒ 2454.83

Round(2454.8989, 2) ⇒ 2454.90

Round(2454.8999, 3) ⇒ 2454.900

Round(2454.5789, -1) ⇒ 2450.5789

Round(2655.5789, -1) ⇒ 2660

Round(3294.8074, 3) ⇒ 3000

Before point
-ve

After point
+ve

2454.7600

12. Trunc() :-

It is similar to round() but it always rounds-off the given number to the lower value.

Syntax :-

trunc(Number, [scale])

Ex :- Trunc(5.3) # 5

Trunc(5.7) # 5

24/08/24

13. months-between ():

'It is used to obtain number of months present between the given two dates'.

Syntax:-

months-between (date1, date2)

EX:-

Select months-between ('24-AUG-24', '24-AUG-25')

from dual;

14. Last-day ():-

'It is used to obtain the last day in the particular of the given date'

Syntax:-

Last-day (date)

EX:- Last-day (sysdate)

from dual;

15. to-char ():-

'It is used to convert the given date into string format based on the model given'.

Syntax:-

to-char (date, format)

Format

model

i, year : TWENTY TWENTY

ii, yyyy : 2020

iii, yy : 20

iv, MONTH : JULY

v, MON : JUL

vi, MM : 07

vii, DAY : Wednesday

viii, DY : :Wed

ix, DD : 08

x, D : 4 (day of the week)

xi, HH24 : 17 hours

xii, HH12 : 5 hours

xiii, MI : 22 minutes

xiv, SS : 53 seconds

xv, 'HH12:MI:SS' : 5:22:53

xvi, 'DD-MM-YY' : 17-05-20

xvii, 'MM-DD-YYYY' : 05-17-2020

Select to-char(hiredate, 'DAY')
from emp;

Select to-char(hiredate, 'MON')
from emp

where to-char(hiredate, 'MON') = 'JAN';

16. NVL (NULL value logic):-

It is used to eliminate the side effects of using null in arithmetic operations.

Syntax:- $NVL(Ag1, Ag2)$

Argument 1:- Here write any column/EXP which can result in null.

Argument 2:- Here we write a numeric value which will be substituted if argument 1 results in NULL, then the 2nd argument value will be considered.

If Argument 1 is NOT NULL then the same value will be considered.

Ex:-

Select sal + NVL(comm, 0)

From emp; o/p:-

Select NVL(NULL, 10)

From emp; o/p:- 10

Select NVL(0, 10)

From DUAL; o/p:- 0

ATTRIBUTES / PROPERTIES

1. key attribute / candidate key:- Any attributes with which we can identify a record.

Uniquely from the table is known as key attribute.

Ex:- SID, MAIL-ID, phone-no.

2. Non key attribute:- All the other attributes except key attribute is known as non key attributes.

Ex:- Sname, Age, Address, DOB, Branch, percentage.

3. prime key attribute:- Among the key attributes an attribute is chosen to be the main attribute to identify a record uniquely from the table.

Ex:- SID

4. Non prime key attribute :- All the key attributes except prime key attribute is known as non prime key attribute.

EX:- Mail-id, ph-no.

5. Composite attribute :- It is combination of 2 or more non key attributes by which we can identify a record uniquely from table. [It is used only when there is no key attribute in the table]

EX:- sname, dob, Address.

6. Super key attribute :- It is a set of all the key attribute

EX:- sid, phone-no, mail-id

7. Foreign key attribute :- It is an attribute which behaves as an key attribute of another table to represent the relationship

EX:- Entity : student

properties : sid, sname, age, address, mail-id, percentage, dob, branch, phone-no

Functional dependency
A relation exists such that an attribute determines another attribute uniquely is known as functional dependency.

Types of functional dependency :-

1. Total functional dependency

2. Partial functional dependency

3. Transitive functional dependency

1. Total functional dependency :-

If all the attributes of a relation are determined a key attribute we call it as total functional dependency.

EX:- let us consider a relation r_1 with four attributes

A, B, C and D. In which A is key attribute.

$$R_1 = \{A, B, C, D\}$$

A^* - key attribute

$$A \twoheadrightarrow B$$

$$A \twoheadrightarrow C$$

$$A \twoheadrightarrow D$$

$$A \twoheadrightarrow (B, C, D)$$

2. partial functional dependency :-

A relation is said to have partial functional dependency if $\rightarrow$ It consists of composite key attribute $\rightarrow$ There exists a dependency such that an attribute determines another attribute

which is a part of composite key attribute

Ex:- let us consider a relation R_2 with four attributes A, B, C and D . in which (A, B) is composite key attribute.

$$R_2 = \{A, B, C, D\}$$

(A, B) - composite key attribute.

$$(A, B) \twoheadrightarrow (C, D)$$

$$B \twoheadrightarrow D$$

3. Transitive functional dependency :-

There exists a dependency such that an attribute is determined by another attribute which is in turn determined by key attribute is known as transitive functional dependency.

Ex:- let us consider a relation R_3 with four attributes A, B, C and D which A is key attribute

$$A \twoheadrightarrow B$$

$$A \rightarrow B$$

$$A \twoheadrightarrow D$$

$$B \rightarrow C$$

$$A \twoheadrightarrow C$$

$$A \rightarrow C$$

Redundancy :- The repetition of unwanted data is known as redundancy.

Anomaly :- The side effects that occur due to dml operation is known as anomaly.

Normalization

It is process of converting a large table into smaller table in order to remove redundancies and anomalies by identifying their functional dependencies is known as Normalization.

OR,

It is process of decomposing a table into normal form is known as normalization.

Normalisation :-

Any table which doesn't have redundancies and anomalies is said to be in normal form.

Levels of Normal Form :-

1st Normal form (1NF)

2nd Normal form (2NF)

3rd Normal form (3NF)

Sachin
Sa

Chauhan
b

Note :- If we reduce any give table given to 3NF, then the table is said to be normalized $j=0$.

1st Normal form (1NF) :-

→ No duplicate records

→ Multivalued data should not be present

2nd Normal form (2NF) :-

→ Table should be in 1NF

→ Table should not have partial functional

→ dependency. It is 1NF

3rd Normal Form (3NF) :-

- Table should be in 2NF
- Table should not have Transitive functional dependency.
- 100% 200% 300% 400% 500%

25/08/24

Attribute examples

→ Attributes :-

1. Name
2. Age
3. ph-no
4. Mail-id
5. Gender
6. Location
7. Height
8. weight
9. Account-no
10. Qualification
11. Aadhar-no
12. pan-no
13. Lic-no
14. Blood group
15. DOB
16. Mothername
17. Fathername

→ Key attributes :-

1. ph-no
2. Mail-id
3. Account-no
4. Aadhar-no
5. pan-no
6. Lic-no

→ Non-key attribute :-

1. Name
2. Age
3. Gender
4. Loc
5. Height
6. weight
7. Qualification
8. Blood group
9. DOB
10. Mothername
11. Fathername

→ primary key attribute :-

1. Aadhar-no

→ Non-primary key attribute :-

1. ph-no
2. Mail-id
3. Account-no
4. pan-no
5. Lic-no

→ composite key attribute :-

1. Name
2. Loc
3. Fathername
4. Mothername

→ combination of 2 or more attributes to find a record uniquely.

→ Super set key :- Set of primary key attributes

→ Photo

→ mailid

→ Account-no

→ pan-no

→ Lic-no

→ Aadhar-no

3. member

2. trader

1. doc

4. dealer

5. wife

1. INWS

10. Export license

10. Import license

5. DAB

8. Brody Jarak

4. Ortophical

1. Import license
2. Export license
3. member
4. dealer
5. wife
6. INWS
7. Ortophical
8. Brody Jarak
9. DAB
10. Import license
10. Export license